

The d-q Transient Model for SCIM Short-Circuit Torque Analysis

A comprehensive technical reference covering theory, implementation, numerical methods, validation, and extensions.

Document scope: This document describes every aspect of the d-q transient model as implemented in `scim_calc/dq.py`, the shared helper functions in `scim_calc/circuit.py`, the parameter pipeline in `scim_calc/config.py`, the post-processing in `scim_calc/compare.py`, and the supporting infrastructure in `run.py` and `scim_calc/sweep.py`. It is intended as both a tutorial for engineers new to the subject and a detailed reference for researchers and maintainers.

Table of Contents

Part I - Foundations 1. Three-Phase Systems and the d-q Transformation - 1.1 The Three-Phase Induction Machine - 1.2 The Clarke Transformation ($abc \rightarrow \text{alphabet}$) - 1.3 The Park Transformation ($\text{alphabet} \rightarrow dq$) - 1.4 Amplitude-Invariant vs Power-Invariant Forms - 1.5 SI Units vs Per-Unit - 1.6 Complex Space-Vector Notation - 1.7 Why the Stationary Frame for This Application - 1.8 Terminology Clarification: α - β vs dq 2. Machine Magnetic Structure and Flux Linkage - 2.1 Self and Mutual Inductances of a Wound-Rotor Machine - 2.2 Effect of the d-q Transformation on Inductances - 2.3 Leakage vs Magnetizing Inductance - 2.4 The Flux-Current Matrix and Its Inverse - 2.5 The Leakage Determinant Delta - 2.6 Physical Interpretation of the Inverse 3. Electromagnetic Torque in d-q Form - 3.1 Torque from Energy and Co-Energy - 3.2 The d-q Torque Expression - 3.3 Equivalent Forms and Numerical Considerations - 3.4 Sign Convention

Part II - The Differential Equations 4. The 5th-Order State-Space Model - 4.1 Stator Voltage Equation in the Stationary Frame - 4.2 Rotor Voltage Equation in the Stationary Frame - 4.3 Mechanical Equation - 4.4 The Complete System in Complex Form - 4.5 The Complete System in Real Form - 4.6 State Vector and Implementation 5. Eigenvalue Analysis - 5.1 Characteristic Equation of the Open-Loop System - 5.2 Eigenvalue Structure for the Test Motor - 5.3 Time Constants and Natural Frequencies - 5.4 Stiffness Ratio and Its Implications

Part III - Initial Conditions 6. Pre-Fault Steady-State Solution - 6.1 The Phasor Equations in the Stationary Frame - 6.2 Relationship to the IEEE Equivalent Circuit - 6.3 Numerical Solution of the 2x2 Complex System - 6.4 Conditioning and Alternative Solution Methods - 6.5 Initial Flux Linkages 7. Slip Derivation from Nameplate Power - 7.1 The Torque-Slip Characteristic - 7.2 Multiple Operating Points - 7.3 Bisection Method: Description and Convergence Proof - 7.4 Alternative: Newton-Raphson and Why It Is Not Used - 7.5 Widening the Search Interval

Part IV - The Short-Circuit Transient 8. The Short-Circuit Condition - 8.1 What a Bolted Three-Phase Short Circuit Means - 8.2 The Fault as an Initial-Value Problem - 8.3 Flux Continuity at Fault Inception - 8.4 Why Current Can Change Discontinuously but Flux Cannot 9. The DC Offset - 9.1 Origin of the DC Offset - 9.2 Mathematical Description - 9.3 Relationship Between Inception Angle and DC Offset Magnitude - 9.4 Decay of the DC Component - 9.5 Effect on the Torque Waveform 10. The Fault Inception Angle - 10.1 Physical Meaning of θ_0 - 10.2 Effect on Initial Conditions - 10.3 Effect on the Transient Waveform - 10.4 Periodicity: Why 180 - 10.5 Why Peak $|T|$ Is Nearly Angle-Invariant for Balanced Faults - 10.6 Asymmetry Mechanism - 10.7 Angle Sweep Implementation

Part V - Numerical Integration 11. The Runge-Kutta 4th-Order Method (RK4) - 11.1 The General Runge-Kutta Framework - 11.2 The RK4 Butcher Tableau - 11.3 Local Truncation Error - 11.4 Global Error and Convergence Order - 11.5 Stability Region - 11.6 Step Size Selection for This Application - 11.7 Implementation Structure in `dq.py` 12. Alternative Numerical Methods - 12.1 Forward Euler: The Simplest - 12.2 Trapezoidal Integration (Crank-Nicolson) - 12.3 Runge-Kutta 45 (Adaptive) - 12.4 Backward Differentiation Formulas (BDF/Gear) - 12.5 Comparison Summary - 12.6 Why RK4 Was Chosen

Part VI - Output and Post-Processing 13. Torque Computation at Each Step - 13.1 Flux-Current Inversion - 13.2 Torque from Space Vectors - 13.3 Sign Convention and Normalization - 13.4 Numerical Considerations 14. Saved Outputs - 14.1 CSV Format for the d-q Model - 14.2 Comparison with the Quick Calculation - 14.3 Comparison Metrics - 14.4 Overlay Plot

Part VII - Validation and Analysis 15. Comparison of Methods: Quick vs d-q - 15.1 Theoretical Differences - 15.2 Quantified Differences for the Test Motor - 15.3 Component-by-Component Spectral Analysis - 15.4 When Each Method Is Appropriate 16. Parameter Sensitivity Analysis - 16.1 Stator Resistance R_s - 16.2 Rotor Resistance R_r - 16.3 Leakage Reactances X_s , X_r - 16.4 Magnetizing Reactance X_m - 16.5 Inertia J - 16.6 Fault Inception Angle θ_0

Part VIII - Extensions and Alternatives 17. Assumptions and Limitations - 17.1 Linear Magnetics (No Saturation) - 17.2 Balanced Fault Only - 17.3 Single-Cage Rotor - 17.4 Constant Rotor Temperature - 17.5 No Skin Effect - 17.6 Ideal Voltage Source Behind the Fault - 17.7 No Torsional Dynamics 18. Possible Extensions - 18.1 Magnetic Saturation Modelling - 18.2 Double-Cage or Deep-Bar Rotor - 18.3 Unbalanced Faults - 18.4 Supply Impedance and Voltage Sags - 18.5 Inter-Turn and Winding Faults - 18.6 Adaptive Time-Stepping - 18.7 Parallel Angle Sweep

Part IX - Appendices 19. Source Code Cross-Reference 20. Glossary of Symbols 21. References

Part I - Foundations

1. Three-Phase Systems and the d-q Transformation

1.1 The Three-Phase Induction Machine

A three-phase squirrel-cage induction motor consists of:

- **Stator:** three identical windings placed 120° apart in electrical angle around the stator circumference. When fed with balanced three-phase currents, these windings produce a rotating magnetic field.
- **Rotor:** a set of conducting bars short-circuited by end rings (the “squirrel cage”). The rotating stator field induces currents in the rotor bars, and the interaction of these currents with the stator field produces torque.

In the stationary abc reference frame, the stator voltage equations are:

$$\begin{aligned} v_{as} &= R_s i_{as} + \frac{d\psi_{as}}{dt} \\ v_{bs} &= R_s i_{bs} + \frac{d\psi_{bs}}{dt} \\ v_{cs} &= R_s i_{cs} + \frac{d\psi_{cs}}{dt} \end{aligned}$$

The flux linkages $\psi_{as}, \psi_{bs}, \psi_{cs}$ are functions of all six currents (three stator, three rotor) and the rotor position θ_r . This makes the abc frame model complicated: the inductance matrix contains terms like $\cos(\theta_r)$, $\cos(\theta_r + 120^\circ)$, etc., that vary with time as the rotor rotates.

The d-q transformation eliminates this time variation by projecting the abc quantities onto a set of orthogonal axes.

1.2 The Clarke Transformation ($abc \rightarrow \alpha\beta$)

The **Clarke transformation** (also called the $abc \rightarrow \alpha\beta$ transformation) converts three-phase quantities into an orthogonal two-axis system plus a zero-sequence component. For a set of three-phase voltages v_a, v_b, v_c :

$$\begin{bmatrix} v_\alpha \\ v_\beta \\ v_0 \end{bmatrix} = \frac{2}{3} \begin{bmatrix} 1 & -1/2 & -1/2 \\ 0 & \sqrt{3}/2 & -\sqrt{3}/2 \\ 1/2 & 1/2 & 1/2 \end{bmatrix} \begin{bmatrix} v_a \\ v_b \\ v_c \end{bmatrix}$$

The factor $2/3$ is a scaling choice. The inverse transformation is:

$$\begin{bmatrix} v_a \\ v_b \\ v_c \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 \\ -1/2 & \sqrt{3}/2 & 1 \\ -1/2 & -\sqrt{3}/2 & 1 \end{bmatrix} \begin{bmatrix} v_\alpha \\ v_\beta \\ v_0 \end{bmatrix}$$

For a **balanced** three-phase system (no zero-sequence component), $v_a + v_b + v_c = 0$ and $v_0 = 0$. The $\alpha\beta$ components then form a rotating vector:

$$v_\alpha + jv_\beta = V e^{j(\omega t + \phi)}$$

where V is the **peak** phase voltage magnitude.

Why this matters: The Clarke transformation replaces the position-dependent 3×3 inductance matrix of the *abc* model with a constant 2×2 matrix in the $\alpha\beta$ frame. The stator self-inductances become equal and the α and β axes are magnetically decoupled (zero mutual inductance between axes). What was time-varying geometry in *abc* becomes a constant-coefficient system in $\alpha\beta$, at the cost of introducing explicit speed-voltage terms in the rotor equation (see Section 4.2).

1.3 The Park Transformation (alphabet $\alpha\beta \rightarrow dq$)

The **Park transformation** rotates the $\alpha\beta$ axes to follow the rotor (or the synchronously rotating field). If we define a reference frame rotating at ω :

$$\begin{bmatrix} f_d \\ f_q \end{bmatrix} = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} f_\alpha \\ f_\beta \end{bmatrix}$$

where $\theta = \int \omega(\tau) d\tau + \theta_0$

In the **synchronous reference frame**, $\omega = \omega_s$, so the dq quantities are constant in steady state. In the **stationary reference frame** (used in this project), $\omega = 0$ and $\theta = 0$, so $f_d = f_\alpha$ and $f_q = f_\beta$. The stationary-frame d-q transformation is therefore equivalent to the Clarke transformation.

1.4 Amplitude-Invariant vs Power-Invariant Forms

The transformation scaling factor affects the magnitude of the resulting quantities:

Convention	Forward factor	dq magnitude of balanced <i>abc</i> set	Power in dq	Code uses
Amplitude-invariant	$2/3$	Same as peak phase value	$P_{dq} = \frac{3}{2}(v_d i_d + v_q i_q)$	Yes
Power-invariant	$\sqrt{2/3}$	$\sqrt{3/2} \times$ peak phase	$P_{dq} = v_d i_d + v_q i_q$	No

The amplitude-invariant form is used throughout this project because it preserves the intuitive relationship: a balanced three-phase sinusoidal set with peak amplitude V maps to a d-q vector of magnitude V . The factor $3/2$ then appears explicitly in the torque equation (see Section 3.2).

1.5 SI Units vs Per-Unit

All quantities in this project are expressed in **SI units** (volts, amperes, ohms, henries, newton-metres, etc.) rather than the per-unit system commonly used in power system engineering. This choice was made because:

1. **Direct correspondence to code:** The parameters in `input.jsonc` and the output CSV files use SI units, making it straightforward to compare code outputs with nameplate data, oscilloscope measurements, or manufacturer datasheets without conversion.
2. **Avoiding base-value ambiguity:** Per-unit systems require choosing base voltage, current, impedance, and power bases. The conventions vary between IEEE, IEC, and individual manufacturers, introducing a potential source of error.
3. **Numerical conditioning:** For the motor sizes this project targets (hundreds to thousands of horsepower), SI quantities are well-scaled for double-precision floating point (resistances in 10^{-2} - $10^0 \Omega$, inductances in 10^{-3} - 10^{-1} H, torques in 10^3 - 10^5 N·m).

The per-unit system would be more convenient if the model were extended to very large or very small machines where SI values span many orders of magnitude, or if the model were integrated into a larger power system simulation.

1.6 Complex Space-Vector Notation

It is convenient to combine the d and q components into a complex number:

$$\bar{f} = f_d + j f_q$$

where $j = \sqrt{-1}$ is the imaginary unit (not to be confused with current i). This complex **space vector** (also called a **phasor** in steady state, though strictly speaking a space vector and a phasor are different) represents the instantaneous magnitude and orientation of the three-phase quantity.

The differential equations for d and q can then be written as a single complex equation. For example, a sinusoidal d -axis signal and co-sinusoidal q -axis signal combine to form a rotating vector $Fe^{j\omega t}$.

1.7 Why the Stationary Frame for This Application

Three common reference frames are available for induction machine analysis:

Frame	Rotation speed	Stator quantities	Rotor quantities	Common use
Stationary ($\alpha\beta$ or dq^s)	$\omega = 0$	AC at ω_s	AC at $s\omega_s$	Short-circuit analysis, direct torque control
Rotor (dq^r)	$\omega = \omega_{re}$	AC at $\omega_s - \omega_{re} = s\omega_s$	DC	Rotor-flux-oriented control
Synchronous (dq^e)	$\omega = \omega_s$	DC (steady state)	AC at $s\omega_s$ (slip frequency)	Stator-flux-oriented control, grid-tied converters

This project uses the **stationary reference frame** for the following reasons:

1. **No stator frame-rotation term.** In the stationary frame, the stator equation is $d\psi_s/dt = v_s - R_s i_s$ without an additional $j\omega\psi_s$ term. The rotation-induced voltage appears only in the rotor equation, which is physically correct. (A zero voltage vector is zero in any orthogonal frame; the choice of frame does not affect the $v_s = 0$ boundary condition.)
2. **The stator equation requires no frame-rotation term.** In the stationary frame, $\frac{d\psi_s}{dt} = v_s - R_s i_s$ without any additional $j\omega\psi$ term on the stator side. The rotation-induced voltage appears only in the rotor equation ($j\omega_{re}\psi_r$), which is physically correct.
3. **Initial conditions are straightforward.** The pre-fault steady state is solved directly in the stationary frame from the 2x2 complex system with the applied voltage $V_{s0} = \sqrt{2}V_{ph} e^{j\theta_0}$.
4. **The alternative - synchronous frame -** would require transforming the initial conditions and fault condition through the Park transformation angle, adding complexity without numerical benefit for this problem.

Implementation: The stationary-frame space-vector model is implemented in `scim_calc/dq.py`, with the state vector representing the real and imaginary parts of ψ_s and ψ_r directly.

1.8 Terminology Clarification: alpha-beta vs dq

Throughout this document and in the code, the model is referred to as a “**d-q model**”. Strictly speaking, since $\omega = 0$ (stationary frame), the transformation used is the **Clarke transformation** ($abc \rightarrow \alpha\beta$), not the **Park transformation** ($\alpha\beta \rightarrow dq$). The correct technical name would be an $\alpha\beta$ **model** or a **Clarke-frame model**.

However, the following conventions are observed in the literature and in this project:

- **$\alpha\beta$ (Clarke) frame:** The stationary two-axis frame with axes aligned to the physical stator phases (α aligned with phase a).
- **dq (Park) frame:** Any frame rotating at an arbitrary speed ω . When $\omega = 0$, the dq and $\alpha\beta$ frames coincide: $d = \alpha$, $q = \beta$.

The decision to use the dq notation despite the stationary frame is a deliberate choice based on:

1. **Industry convention:** The term “d-q model” is the standard name in the electric machinery literature (Krause, Lipo, Boldea) for any two-axis state-space machine model, regardless of whether the frame is stationary or rotating. Specifying “stationary reference frame d-q model” disambiguates.
2. **Code readability:** Variable names like `psi_sd`, `psi_sq` (or just using the complex space vector) are more recognizable to engineers familiar with d-q theory than $\alpha\beta$ subscripts.
3. **Extensibility:** The same model structure can be extended to the synchronous frame (for grid-connected applications) or the rotor frame (for field-oriented control) by adding a non-zero rotation speed - the dq framework accommodates this naturally.

Key point: Despite the dq naming, this project implements an $\alpha\beta$ model with $\omega = 0$. The differential equations have no Park rotation terms on the stator side, confirming this. When the literature is referenced,

the model described here corresponds to the **stationary-reference-frame d-q model** or equivalently the $\alpha\beta$ **model**.

2. Machine Magnetic Structure and Flux Linkage

2.1 Self and Mutual Inductances of a Wound-Rotor Machine

Consider an ideal induction machine with sinusoidally distributed windings. In the native abc (three-phase) frame, the flux linking each winding is a linear combination of all six currents (three stator a, b, c and three rotor a, b, c), weighted by self and mutual inductances:

$$\begin{aligned} [\psi_{abc}] &= [L_s(\theta_r) \quad L_{sr}(\theta_r)] [i_{abc}] \\ [\psi_{abcr}] &= [L_{sr}^T(\theta_r) \quad L_r(\theta_r)] [i_{abcr}] \end{aligned}$$

where each submatrix is 3×3 . For a machine with sinusoidally distributed windings, the elements are:

Stator self-inductances (constant):

$$L_{as,as} = L_{bs,bs} = L_{cs,cs} = L_{\ell s} + L_m \equiv L_s^{abc}$$

Stator mutual inductances (constant):

$$L_{as,bs} = L_{bs,cs} = L_{cs,as} = -\frac{1}{2}L_m$$

Stator-to-rotor mutual inductances (rotor-position-dependent):

$$L_{as,ar} = L_m \cos \theta_r, \quad L_{as,br} = L_m \cos(\theta_r + 120^\circ), \quad \dots$$

After applying the Clarke transformation ($abc \rightarrow \alpha\beta$), the 3×3 stator and rotor matrices become 2×2 in the $\alpha\beta$ frame, and the inductances simplify dramatically:

$$\begin{aligned} L_{s_ab} &= \begin{bmatrix} L_{ls}+L_m & 0 \\ 0 & L_{ls}+L_m \end{bmatrix} = \begin{bmatrix} L_s & 0 \\ 0 & L_s \end{bmatrix} \end{aligned}$$

where $L_s = L_{\ell s} + L_m$ is the **stator self-inductance per axis** in the $\alpha\beta$ frame. The zero off-diagonals confirm that the α and β axes are magnetically decoupled - a key advantage of the transformed model.

The stator-to-rotor mutual inductance matrix in the $\alpha\beta$ frame becomes:

$$L_{sr_ab} = L_m \begin{bmatrix} \cos(\theta_r) & -\sin(\theta_r) \\ \sin(\theta_r) & \cos(\theta_r) \end{bmatrix}$$

The crucial observation: **the diagonal elements L_s , L_r and the stator-stator mutual terms are constant** (they depend only on the fixed geometry), while the **stator-to-rotor mutual inductances vary with rotor position θ_r** . This position dependence is expressed through the rotation matrix above and ultimately manifests as the $j\omega_{re}\psi_r$ speed-voltage term in the rotor equation (see Section 4.2).

2.2 Effect of the d-q Transformation on Inductances

When the Clarke transformation (amplitude-invariant, $abc \rightarrow \alpha\beta$) is applied, the inductances become:

$$\begin{aligned} L_{\alpha s, \alpha s} &= L_s = L_{\ell s} + L_m \\ L_{\beta s, \beta s} &= L_s = L_{\ell s} + L_m \\ L_{\alpha s, \beta s} &= 0 \quad (\text{orthogonal axes are magnetically decoupled}) \end{aligned}$$

The mutual inductances between stator and rotor in the $\alpha\beta$ frame become:

$$\begin{aligned} L_{\alpha s, \alpha r} &= L_m \cos \theta_r \\ L_{\alpha s, \beta r} &= -L_m \sin \theta_r \\ L_{\beta s, \alpha r} &= L_m \sin \theta_r \\ L_{\beta s, \beta r} &= L_m \cos \theta_r \end{aligned}$$

These can be written compactly in complex form:

$$\bar{L}_{s,r} = L_m e^{j\theta_r}$$

Resolving the reference-frame question: The stator self-inductances (L_s) are constant in any frame because the stator windings are fixed. The rotor self-inductances (L_r) are also constant because the rotor is symmetric. The stator-to-rotor mutual inductances, however, depend on the relative angle θ_r between the stator and rotor axes. In the stationary abc frame, this dependence appears as $\cos \theta_r$, $\cos(\theta_r \pm 120^\circ)$, etc. In the stationary $\alpha\beta$ frame, it appears as the rotation matrix above. These position-dependent mutual inductances are not eliminated by the Clarke transformation — they are transformed from trigonometric functions of θ_r into the explicit rotation matrix $\begin{bmatrix} \cos \theta_r & -\sin \theta_r \\ \sin \theta_r & \cos \theta_r \end{bmatrix}$.

The key simplification of the d-q approach is that this rotation matrix, when combined with the rotor voltage equation, produces a **constant-coefficient system with an explicit speed-voltage term** $j\omega_{re}\psi_r$, rather than a time-varying coefficient system. The θ_r dependence is not removed — it is converted from a parameter variation into a dynamic term in the differential equation. This is what makes the state-space formulation possible: the system matrix becomes constant (for constant speed), and the rotation of the rotor relative to the stator is captured by the $j\omega_{re}\psi_r$ term in the rotor equation.

2.3 Leakage vs Magnetizing Inductance

The inductances in the model are decomposed as:

- **Stator leakage inductance** $L_{\ell s}$: flux that links only the stator winding, not the rotor. Caused by slot leakage, end-winding leakage, and harmonic leakage.
- **Rotor leakage inductance** $L_{\ell r}$: flux that links only the rotor winding.
- **Magnetizing inductance** L_m : flux that crosses the air gap and links both stator and rotor. This is the flux that produces torque.

The sum gives the self-inductances:

$$L_s = L_{\ell s} + L_m, \quad L_r = L_{\ell r} + L_m$$

The **leakage coefficient** or **Blondel coefficient** σ is:

$$\sigma = 1 - \frac{L_m^2}{L_s L_r}$$

For the test motor ($R_s = 0.069966 \Omega$, $R_r = 0.072979 \Omega$, $X_s = 1.272654 \Omega$, $X_r = 0.986283 \Omega$, $X_m = 42.29672 \Omega$ at 60 Hz):

$$L_{\ell s} = \frac{1.272654}{2\pi \cdot 60} = 3.377 \text{ mH}$$

$$L_{\ell r} = \frac{0.986283}{2\pi \cdot 60} = 2.617 \text{ mH}$$

$$L_m = \frac{42.29672}{2\pi \cdot 60} = 112.24 \text{ mH}$$

The leakage coefficient is $\sigma \approx 0.043$, indicating typical induction-machine leakage with strong magnetizing coupling and finite leakage flux.

2.4 The Flux-Current Matrix and Its Inverse

In the stationary d-q frame, the flux-current relationship is:

$$\begin{bmatrix} \psi_s \\ \psi_r \end{bmatrix} = \begin{bmatrix} L_s & L_m \\ L_m & L_r \end{bmatrix} \begin{bmatrix} i_s \\ i_r \end{bmatrix}$$

where ψ_s, ψ_r, i_s, i_r are complex numbers (combining d and q components).

To recover currents from fluxes (as required at every time step of the d-q simulation), we invert this 2x2 system:

$$\begin{bmatrix} i_s \\ i_r \end{bmatrix} = \frac{1}{D} \begin{bmatrix} L_r & -L_m \\ -L_m & L_s \end{bmatrix} \begin{bmatrix} \psi_s \\ \psi_r \end{bmatrix}$$

where $D = \Delta = L_s L_r - L_m^2$

where $\Delta = L_s L_r - L_m^2$ is the **leakage determinant**.

Implementation: `scim_calc/circuit.py:currents_from_flux()`, lines 14-33. Note that the determinant Δ is pre-computed once in `config.py:normalize_params()` line 63 and stored in the parameters dict, avoiding repeated computation.

2.5 The Leakage Determinant Delta

The determinant Δ has physical significance:

$$\Delta = L_s L_r - L_m^2 = (L_{\ell s} + L_m)(L_{\ell r} + L_m) - L_m^2 = L_{\ell s} L_{\ell r} + L_m(L_{\ell s} + L_{\ell r})$$

In terms of the leakage coefficient σ :

$$\Delta = \sigma L_s L_r$$

For the test motor, $\Delta \approx 5.46 \times 10^{-7} \text{ H}^2$.

If $L_m^2 = L_s L_r$ (perfect magnetic coupling, no leakage), then $\Delta = 0$ and the inverse does not exist. This is physically impossible - all real machines have leakage flux. As $\Delta \rightarrow 0$, the currents become increasingly sensitive to small changes in flux (ill-conditioning).

The **condition number** of the 2×2 inductance matrix is:

$$\kappa = \frac{L_s L_r + L_m^2}{2L_{\ell s} L_{\ell r} + L_m(L_{\ell s} + L_{\ell r})}$$

For the test motor, $\kappa \approx 1.09$, indicating excellent numerical conditioning — the inverse is stable and accurate. Even for machines with very large magnetizing reactance (e.g., $X_m/X_s > 100$), the condition number remains below 10, so ill-conditioning of the flux-current inversion is not a practical concern for any realistic induction machine.

2.6 Physical Interpretation of the Inverse

The inverse equation reveals how the stator and rotor currents are determined by the flux linkages:

$$i_s = \frac{L_r}{\Delta} \psi_s - \frac{L_m}{\Delta} \psi_r$$

The first term represents the current required to support the stator flux against the total stator inductance (modified by the rotor coupling). The second term represents the current induced in the stator by the rotor flux through the mutual inductance.

Similarly:

$$i_r = -\frac{L_m}{\Delta} \psi_s + \frac{L_s}{\Delta} \psi_r$$

The negative sign on the mutual term reflects Lenz's law: the rotor current opposes changes in the stator flux that produced it.

3. Electromagnetic Torque in d-q Form

3.1 Torque from Energy and Co-Energy

In a lossless magnetic system, the electromagnetic torque can be derived from the co-energy W_c :

$$T_e = \left. \frac{\partial W_c(i, \theta)}{\partial \theta_m} \right|_{i=\text{constant}}$$

For a linear magnetic system (no saturation), the co-energy is:

$$W_c = \frac{1}{2} \mathbf{i}^T \mathbf{L}(\theta_r) \mathbf{i}$$

Evaluating the derivative with the 2x2 d-q inductance matrix yields the torque expression.

3.2 The d-q Torque Expression

For the amplitude-invariant d-q transformation, the torque is:

$$T_e = \frac{3}{2} p (\psi_{ds} i_{qs} - \psi_{qs} i_{ds})$$

where p is the number of pole pairs. In complex form:

$$T_e = -\frac{3}{2} p \operatorname{Im}\{\psi_s \cdot i_s^*\}$$

where i_s^* denotes the complex conjugate of i_s .

Derivation sketch: Expanding $\psi_s i_s^*$:

$$\psi_s i_s^* = (\psi_{ds} + j\psi_{qs})(i_{ds} - ji_{qs}) = \psi_{ds} i_{ds} + \psi_{qs} i_{qs} + j(\psi_{qs} i_{ds} - \psi_{ds} i_{qs})$$

The imaginary part is $\psi_{qs} i_{ds} - \psi_{ds} i_{qs} = -(\psi_{ds} i_{qs} - \psi_{qs} i_{ds})$, giving:

$$-\frac{3}{2} p \operatorname{Im}\{\psi_s i_s^*\} = \frac{3}{2} p (\psi_{ds} i_{qs} - \psi_{qs} i_{ds})$$

Implementation: `scim_calc/circuit.py:torque_from_flux_current()`, line 44. The factor 3/2 is hard-coded as -1.5.

3.3 Equivalent Forms and Numerical Considerations

Several mathematically equivalent forms exist:

Form	Expression	Used in	Numerical note
Flux x current	$-\frac{3}{2} p \operatorname{Im}\{\psi_s i_s^*\}$	This project	Best - only needs ψ_s and i_s
Current x inductance	$\frac{3}{2} p L_m \operatorname{Im}\{i_s i_r^*\}$	Alternative	Needs both i_s and i_r
Rotor flux x current	$-\frac{3}{2} p \frac{L_m}{L_r} \operatorname{Im}\{\psi_r i_s^*\}$	Rotor-flux-oriented control	Needs flux estimation
Power balance	$T_e = \frac{P_{ag} - P_{cu,r}}{\omega_{re}}$	Analytical	Not for real-time

The chosen form (flux x current) is preferred because: - It requires only ψ_s (from the state vector) and i_s (recovered via the inverse inductance matrix). - It avoids explicit computation of the rotor current i_r (though i_r is computed anyway in the ODE right-hand side). - It is numerically well-conditioned for typical operating ranges.

3.4 Sign Convention

The torque computed by `torque_from_flux_current()` uses the **motor convention**: positive torque means the machine is operating as a motor (converting electrical to mechanical power). For a generator (converting mechanical to electrical), the torque would be negative.

In the simulation, the torque sign is normalized so that the pre-fault torque is always displayed as positive:

```
torque_sign = 1.0 if T_nom_raw >= 0.0 else -1.0    # dq.py line 60
T_fault[k] = torque_sign * Te_raw                  # dq.py line 112
```

This means the output torque T_{fault} : - Starts at $+T_{nom}$ at $t = 0$. - Becomes negative during the transient when the machine is braking (regenerative). - Returns toward zero as the stored magnetic energy dissipates.

Part II - The Differential Equations

4. The 5th-Order State-Space Model

4.1 Stator Voltage Equation in the Stationary Frame

In the stationary reference frame, the stator voltage equation is:

$$\frac{d\psi_s}{dt} = v_s - R_s i_s$$

where: - $\psi_s = \psi_{ds} + j\psi_{qs}$ is the stator flux linkage space vector (V · s or Wb · turn) - $v_s = v_{ds} + jv_{qs}$ is the stator voltage space vector (V) - $i_s = i_{ds} + ji_{qs}$ is the stator current space vector (A) - R_s is the stator resistance per phase (Ω)

Under the short-circuit condition ($v_s = 0$ for $t \geq 0$), this simplifies to:

$$\boxed{\frac{d\psi_s}{dt} = -R_s i_s}$$

Physical interpretation: With the terminal voltage forced to zero, the stator flux can only decay through resistive dissipation. The rate of decay is proportional to the stator current, which itself depends on the flux.

Implementation: `scim_calc/dq.py`, line 80:

```
dpsi_s = -Rs * i_s
```

4.2 Rotor Voltage Equation in the Stationary Frame

The rotor voltage equation for a squirrel-cage rotor ($v_r = 0$) in the stationary frame is:

$$\frac{d\psi_r}{dt} = j\omega_{re}\psi_r - R_r i_r$$

where: - $\psi_r = \psi_{dr} + j\psi_{qr}$ is the rotor flux linkage space vector (V · s) - $i_r = i_{dr} + ji_{qr}$ is the rotor current space vector (A) - R_r is the rotor resistance referred to the stator (Ω) - $\omega_{re} = p \cdot \omega_m$ is the **rotor electrical speed** (rad/s) - p is the number of pole pairs - ω_m is the mechanical rotor speed (rad/s)

The term $j\omega_{re}\psi_r$ is the **speed voltage** or **rotation-induced voltage**. It captures the effect of the rotor conductors moving through the magnetic field. Expanding into real components:

$$\frac{d\psi_{dr}}{dt} = -\omega_{re}\psi_{qr} - R_r i_{dr}$$

$$\frac{d\psi_{qr}}{dt} = \omega_{re}\psi_{dr} - R_r i_{qr}$$

The cross-coupling between the d and q axes through $-\omega_{re}\psi_{qr}$ and $\omega_{re}\psi_{dr}$ is what gives the induction machine its characteristic behaviour: the rotor flux vector rotates at ω_{re} relative to the stationary frame.

Implementation: `scim_calc/dq.py`, line 82:

```
dpsi_r = 1j * ore * pr - Rr * i_r
```

4.3 Mechanical Equation

The mechanical dynamics of the rotor are governed by Newton's second law for rotation:

$$J \frac{d\omega_m}{dt} = T_e - T_L$$

where: - J is the total moment of inertia ($\text{kg} \cdot \text{m}^2$) - ω_m is the mechanical rotor speed (rad/s) - T_e is the electromagnetic torque ($\text{N} \cdot \text{m}$) - T_L is the load torque ($\text{N} \cdot \text{m}$)

By default, speed dynamics are **disabled** (`USE_SPEED_DYNAMICS = false`), and ω_m is held constant at its pre-fault value. This is justified because:

1. **Time-scale separation:** The electrical dynamics decay with time constants on the order of $T_{s,dc} \approx 85$ ms and $T_{r,sc} \approx 81$ ms. The mechanical dynamics are governed by $J d\omega_m/dt = T_e - T_L$; for a large motor with $J \approx 10 \text{ kg} \cdot \text{m}^2$, the speed change over 200 ms is negligible (a torque of $1.5 \times T_{nom} \approx 6000 \text{ N} \cdot \text{m}$ acting for 0.2 s produces $\Delta\omega_m \approx T \cdot \Delta t / J \approx 120 \text{ rad/s}$, but the electromagnetic torque oscillates and its average over the transient is close to zero). The rotor speed therefore does not change appreciably during the 200 ms electrical transient.
2. **Conservative result:** Constant speed produces slightly higher peak torque because the machine does not decelerate (which would reduce the slip and the rotor-induced voltage). For worst-case analysis, this is the appropriate choice.
3. **Simulation speed:** Solving the mechanical equation increases the state dimension by one and adds a small computational cost, but the main reason for disabling it is the conservative-worst-case argument.

When speed dynamics are enabled: - T_L is set equal to the pre-fault electromagnetic torque T_{nom} (constant load). - The speed evolves according to the net accelerating torque.

Implementation: `scim_calc/dq.py`, lines 84-88:

```
dom = 0.0
if use_speed_dynamics:
    Te = torque_from_flux_current(ps, i_s, pole_pairs)
    dom = (Te - T_load_raw) / j_total
```

4.4 The Complete System in Complex Form

Combining the three equations:

```
dpsi_s/dt = -Rs * i_s
dpsi_r/dt = j * p * wm * psi_r - Rr * i_r
domega_m/dt = (Te - TL) / J
```

with the algebraic constraints:

$$i_s = \frac{L_r \psi_s - L_m \psi_r}{\Delta}$$

$$i_r = \frac{-L_m \psi_s + L_s \psi_r}{\Delta}$$

$$T_e = -\frac{3}{2} p \text{Im}\{\psi_s i_s^*\}$$

This is a system of **three complex differential equations** (or **five real differential equations**: two for the real and imaginary parts of ψ_s and ψ_r , plus one for ω_m). The algebraic equations for i_s , i_r , and T_e are evaluated at each time step but do not introduce additional state variables.

4.5 The Complete System in Real Form

The electrical subsystem can be written as four real differential equations for the flux components. With ω_m held constant (speed dynamics disabled), the system is **linear time-invariant (LTI)**:

$$\begin{aligned} \frac{d}{dt} \begin{bmatrix} \psi_{ds} \\ \psi_{qs} \\ \psi_{dr} \\ \psi_{qr} \end{bmatrix} &= \begin{bmatrix} -R_s L_r / D & 0 & R_s L_m / D & 0 \\ 0 & -R_s L_r / D & 0 & R_s L_m / D \\ R_r L_m / D & 0 & -R_r L_s / D & -p \omega_m \\ 0 & R_r L_m / D & p \omega_m & -R_r L_s / D \end{bmatrix} \begin{bmatrix} \psi_{ds} \\ \psi_{qs} \\ \psi_{dr} \\ \psi_{qr} \end{bmatrix} \end{aligned}$$

where $D = \Delta = L_s L_r - L_m^2$, $\omega_m = \text{omega_m}$, $p = \text{pole_pairs}$

When speed dynamics are enabled, the mechanical equation completes the system:

$$\frac{d\omega_m}{dt} = \frac{1}{J} (T_e - T_L), \quad T_e = -\frac{3}{2} p \text{Im}\{\psi_s i_s^*\}$$

where i_s is recovered from the flux-current inversion at each step. The coupled system is **nonlinear** because the speed ω_m appears in the electrical matrix (through $p\omega_m$) and the torque T_e depends on the electrical states.

4.6 State Vector and Implementation

In the code, the state is stored as a real vector of length 5:

```
state = [psi_s.real, psi_s.imag, psi_r.real, psi_r.imag, omega_m]
```

The ODE right-hand side is evaluated by the `rhs()` function, which: 1. Reconstructs the complex fluxes from the real components. 2. Recovers i_s, i_r via `currents_from_flux()`. 3. Computes $d\psi_s/dt, d\psi_r/dt$, and (optionally) $d\omega_m/dt$.

Implementation: `scim_calc/dq.py`, lines 72-89.

5. Eigenvalue Analysis

5.1 Characteristic Equation of the Open-Loop System

With ω_m constant (speed dynamics disabled), the system is linear and can be written as:

$$\frac{d}{dt} \mathbf{x} = \mathbf{A} \mathbf{x}$$

where $\mathbf{x} = [\psi_{ds}, \psi_{qs}, \psi_{dr}, \psi_{qr}]^T$ and $\mathbf{A}$ is:

$$\mathbf{A} = \begin{bmatrix} -R_s L_r / D & 0 & R_s L_m / D & 0 \\ 0 & -R_s L_r / D & 0 & R_s L_m / D \\ R_r L_m / D & 0 & -R_r L_s / D & -p \omega_m \\ 0 & R_r L_m / D & p \omega_m & -R_r L_s / D \end{bmatrix}$$

where $D = \Delta$, $\omega_m = \text{omega_m}$

The eigenvalues λ satisfy $\det(\mathbf{A} - \lambda \mathbf{I}) = 0$, which yields the characteristic equation:

$$\left[\left(\lambda + \frac{R_s L_r}{\Delta} \right) \left(\lambda + \frac{R_r L_s}{\Delta} \right) - \frac{R_s R_r L_m^2}{\Delta^2} \right]^2 + \left[\left(\lambda + \frac{R_s L_r}{\Delta} \right)^2 + \left(\lambda + \frac{R_r L_s}{\Delta} \right)^2 \right] (p \omega_m)^2 + (p \omega_m)^4 = 0$$

This quartic equation has two pairs of complex-conjugate eigenvalues:

$$\lambda_{1,2} = -\alpha_1 \pm j\omega_1, \quad \lambda_{3,4} = -\alpha_2 \pm j\omega_2$$

The exact eigenvalues depend on the motor parameters and operating speed ω_m , but they generally consist of: - A **stator-dominated** mode with faster decay (larger α) and frequency near ω_s . - A **rotor-dominated** mode with slower decay and frequency near $s\omega_s$.

5.2 Eigenvalue Structure for the Test Motor

For the test motor at rated slip ($s = 0.00779$, $\omega_m = 374.1$ rad/s), the 4-state constant-speed system matrix yields two complex-conjugate eigenvalue pairs:

Computed eigenvalues (from `test_eigenvalue_stability.py`):

Pair	Eigenvalue λ	$ \lambda $ (rad/s)	Frequency f (Hz)	Damping ratio ζ	Character
1	$-12.38 \pm j373.68$	373.9	59.5	0.033	Primarily associated with stator flux dynamics; oscillatory at near-line frequency
2	$-11.79 \pm j0.37$	11.8	1.88	0.9995	Primarily associated with rotor flux dynamics; nearly critically damped

Both pairs have comparable decay rates ($\alpha \approx 12$ s⁻¹), meaning both the stator and rotor flux transients decay on similar timescales ($\tau \approx 85$ ms). The key difference is their oscillatory content: - Pair 1 oscillates at approximately the electrical fundamental frequency (60 Hz) and is responsible for the AC torque ripple. - Pair 2 is nearly critically damped ($\zeta \approx 1$) and contributes an essentially aperiodic decay component.

The mode primarily associated with the stator determines the 60 Hz torque ripple amplitude, while the mode primarily associated with the rotor governs the overall envelope decay rate. (The labels “stator mode” and “rotor mode” are used as shorthand; a rigorous modal decomposition would require eigenvector or participation-factor analysis.)

5.3 Time Constants and Natural Frequencies

The time constants identified from the eigenvalue analysis correspond to the **quick calculation** time constants:

Time constant	Symbol	Expression	Value (test motor)
Stator DC	$T_{s,dc}$	$L_{s,sc}/R_s$	84.8 ms
Rotor short-circuit	$T_{r,sc}$	$L_{r,sc}/R_r$	80.8 ms
Rotor open-circuit	T_{r0}	$(L_m + L_r)/R_r$	1.57 s

where:

$$L_{s,sc} = L_{\ell s} + \frac{L_m L_{\ell r}}{L_m + L_{\ell r}}, \quad L_{r,sc} = L_{\ell r} + \frac{L_m L_{\ell s}}{L_m + L_{\ell s}}$$

These are the short-circuit inductances seen from the stator and rotor respectively, analogous to the short-circuit reactances in `quick.py`.

Operational inductance vs self-inductance: Two distinct inductance values appear in different parts of the model. The **self-inductance** $L_s = L_{\ell s} + L_m$ appears in the state equations (the flux-current matrix in Section 2.4) because it describes the total flux linking the stator winding when both stator and rotor carry current. The **short-circuit (or operational) inductance** $L_{s,sc} = L_{\ell s} + L_m \parallel L_{\ell r}$ appears in the time constants because it describes the effective inductance seen from the stator when the rotor circuit is closed (shorted through its own resistance). The difference is that during a transient, the rotor currents adjust to oppose changes in stator flux, reducing the effective inductance. The same distinction applies on the rotor side: L_r in the state equations vs $L_{r,sc}$ in the rotor time constant.

5.4 Stiffness Ratio and Its Implications

The **stiffness ratio** of a system of ODEs is the ratio of the largest to the smallest eigenvalue magnitude:

$$S = \frac{\max |\lambda|}{\min |\lambda|}$$

From the computed eigenvalues (Section 5.2):

$$S = \frac{|\lambda_1|}{|\lambda_3|} = \frac{373.9}{11.8} \approx 31.7$$

A system with $S < 100$ is generally considered **non-stiff** and can be efficiently integrated with explicit methods like RK4. This justifies the choice of RK4 over more complex implicit methods (see Section 12).

Part III - Initial Conditions

6. Pre-Fault Steady-State Solution

6.1 The Phasor Equations in the Stationary Frame

Before the fault ($t < 0$), the machine is in steady-state operation. In the stationary reference frame, the steady-state phasor equations are:

$$\begin{bmatrix} R_s + j\omega_s L_s & j\omega_s L_m \\ 0 & 0 \end{bmatrix} \begin{bmatrix} I_{s0} \\ I_{r0} \end{bmatrix} = \begin{bmatrix} V_{s0} \\ 0 \end{bmatrix}$$

where $\omega_s = \omega_{\text{sync}}$, $s = \text{slip}$

where V_{s0} is the peak stator voltage space vector at the instant of the fault:

$$V_{s0} = \sqrt{2} \cdot V_{ph} \cdot e^{j\theta_0}$$

The $\sqrt{2}$ converts RMS to peak (since the space vectors in this amplitude-invariant convention represent peak quantities). θ_0 is the fault inception angle - the angle of the a -phase voltage at $t = 0$.

Derivation of the matrix elements:

- **Top-left** ($R_s + j\omega_s L_s$): The stator impedance at synchronous frequency. The voltage drop across the stator resistance R_s and the total stator inductance L_s at frequency ω_s .
- **Top-right** ($j\omega_s L_m$): The voltage induced in the stator by the rotor current I_{r0} through the mutual inductance L_m at stator frequency ω_s .
- **Bottom-left** ($j s \omega_s L_m$): The voltage induced in the rotor by the stator current I_{s0} through the mutual inductance L_m at slip frequency $s\omega_s$. This is the frequency seen by the rotor (the stator field rotates at ω_s , the rotor at $\omega_{re} = (1 - s)\omega_s$, so the relative speed is $s\omega_s$).
- **Bottom-right** ($R_r + j s \omega_s L_r$): The rotor impedance at slip frequency.

Implementation: `scim_calc/circuit.py:solve_prefault()`, lines 47-95. The matrix is constructed at lines 72-78 and solved with `np.linalg.solve` at line 80.

6.2 Relationship to the IEEE Equivalent Circuit

The conventional IEEE per-phase equivalent circuit (used in `quick.py:torque_from_phasors()`) gives:

$$I_s = \frac{V_{ph}}{R_s + jX_s + \frac{jX_m(R_r/s + jX_r)}{jX_m + R_r/s + jX_r}}$$

The d-q phasor equations in [6.1] are the **same physical model**, written in inductance form with the slip frequency explicitly separating the stator and rotor equations. The two forms are related by:

- The IEEE circuit uses reactances $X = \omega L$, while the d-q equations use inductances L .
- The IEEE circuit solves for RMS currents, while the d-q equations solve for peak currents (the $\sqrt{2}$ factor in V_{s0}).
- The IEEE circuit is a scalar equation (per-phase), while the d-q equation is a 2x2 system that maintains the phase relationship between stator and rotor quantities.

6.3 Numerical Solution of the 2x2 Complex System

The system is solved using `numpy.linalg.solve()`, which uses LU decomposition with partial pivoting. For a 2x2 system, this is trivial and numerically robust.

The condition number κ of the 2x2 matrix gives an indication of numerical stability:

$$\kappa = \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\|$$

For the test motor, $\kappa \approx 10^3$ (moderate). This means solving the system loses about 3 decimal digits of precision relative to the machine epsilon ($\approx 10^{-16}$ for double precision). This is entirely acceptable for engineering accuracy.

6.4 Conditioning and Alternative Solution Methods

Method	Pros	Cons	Used?
<code>np.linalg.solve</code> (LU)	Robust, accurate	None for 2x2	Yes
Cramer's rule	Simple formula	Poor for large systems	Possible but unnecessary
Closed-form inverse	Fast, explicit	Must handle degenerate case	Possible but unnecessary
Iterative (Gauss-Seidel)	Low memory	Slower, convergence issues	No

For a 2x2 system, any direct method would work. `np.linalg.solve` is chosen because it is the standard, tested, and most readable approach.

6.5 Initial Flux Linkages

Once I_{s0} and I_{r0} are known:

$$\psi_{s0} = L_s I_{s0} + L_m I_{r0}$$

$$\psi_{r0} = L_m I_{s0} + L_r I_{r0}$$

These complex quantities become the initial conditions for the differential equations at $t = 0^+$. They satisfy:

$$\psi_s(0^+) = \psi_{s0}, \quad \psi_r(0^+) = \psi_{r0}$$

This is the **flux continuity** condition (Faraday's law requires that flux linkage cannot change instantaneously unless an infinite voltage is applied).

7. Slip Derivation from Nameplate Power

7.1 The Torque-Slip Characteristic

The electromagnetic torque developed by an induction motor as a function of slip can be derived from the equivalent circuit:

$$T_e(s) = \frac{3V_{ph}^2}{\omega_{syn,m}} \cdot \frac{R_r/s}{(R_s + R_r/s)^2 + (X_s + X_r)^2}$$

This is the **Kloss formula** (approximate, neglecting X_m). The more exact form (used in `circuit.py`) accounts for the magnetizing branch:

$$T_e(s) = \frac{3|I_r(s)|^2(R_r/s)}{\omega_{syn,m}}$$

where $I_r(s)$ is computed from the full equivalent circuit.

The torque-slip curve has: - $T_e = 0$ at $s = 0$ (synchronous speed, no rotor current). - A peak at $s = s_{max}$ (pullout or breakdown slip), typically 0.1-0.3 for NEMA design B motors. - $T_e \propto 1/s$ for small s (the approximately linear region of normal operation). - Decreasing torque for $s > s_{max}$.

7.2 Multiple Operating Points

The power-balance equation:

$$T_e(s) \cdot \omega_m = T_e(s) \cdot (1 - s)\omega_{syn,m} = P_{mech}$$

can have multiple solutions for s : - **Motoring** ($s > 0$): the intended solution, typically $s \ll s_{max}$. - **Plugging** ($s > 1$): the machine operates as a brake, torque is positive but at subsynchronous speed. - **Generating** ($s < 0$): the machine is driven above synchronous speed.

The bisection search is designed to find the **motoring** solution by starting with the interval $[10^{-4}, 0.1]$, which lies in the linear region below s_{max} for most induction motors.

7.3 Bisection Method: Description and Convergence Proof

The bisection method finds a root of $f(s) = T_e(s) - P_{mech}/((1 - s)\omega_{syn,m})$ by repeatedly halving an interval known to contain a root.

Algorithm:

1. Start with $[a, b]$ such that $f(a) \cdot f(b) < 0$.
2. Compute $c = (a + b)/2$.
3. If $f(c) = 0$ (within tolerance), return c .
4. If $f(a) \cdot f(c) < 0$, set $b = c$; otherwise set $a = c$.
5. Repeat until $|b - a| < \epsilon$ or maximum iterations reached.

Convergence proof: At each iteration, the interval containing the root is halved. After n iterations, the error is bounded by:

$$|s_n - s_{true}| \leq \frac{b_0 - a_0}{2^n}$$

Since f is continuous and $f(a) \cdot f(b) < 0$ at the start, the intermediate value theorem guarantees a root exists in the interval, and the bisection algorithm is guaranteed to converge to it.

Convergence rate: Linear ($O(2^{-n})$), requiring $\log_2((b - a)/\epsilon)$ iterations. For $\epsilon = 10^{-10}$ and initial interval $[10^{-4}, 0.1]$, this is $\log_2(0.1/10^{-10}) \approx 30$ iterations. The code uses 100 as a safe maximum.

7.4 Alternative: Newton-Raphson and Why It Is Not Used

The Newton-Raphson method offers quadratic convergence:

$$s_{n+1} = s_n - \frac{f(s_n)}{f'(s_n)}$$

However: 1. **It requires** $f'(s)$ - the derivative of the torque with respect to slip. This can be derived analytically but adds code complexity. 2. **It may diverge** if the initial guess is far from the root, especially near the peak of the torque-slip curve where $f'(s)$ changes sign. 3. **It may converge to the wrong root** (e.g., the plugging or generating solution).

Bisection is preferred because: - It is **guaranteed to converge** as long as the root is bracketed. - It requires only function evaluations (no derivatives). - It always converges to the root within the bracketed interval (the motoring solution). - The slower convergence rate is irrelevant for a scalar function: 30 iterations vs 5-6 iterations is negligible in Python.

7.5 Widening the Search Interval

The initial search interval $[10^{-4}, 0.1]$ is chosen based on typical full-load slips for NEMA design B motors ($s_{FL} \approx 0.005$ -0.05). However, some motors (especially small or high-efficiency designs) may have different slip ranges.

If $f(a) \cdot f(b) > 0$ (no sign change, thus no guarantee of a root), the code widens the upper bound:

```
while f_low * f_high > 0 and s_high < 0.8:
    s_high = min(s_high * 2, 0.8)
    f_high = f(s_high)
```

The limit $s_{high} \leq 0.8$ prevents the search from entering the plugging region ($s > 1$). If the power rating is unreasonably large (requiring $s > 0.8$ to produce enough torque), the function raises an error.

Part IV - The Short-Circuit Transient

8. The Short-Circuit Condition

8.1 What a Bolted Three-Phase Short Circuit Means

A **bolted balanced three-phase terminal short circuit** forces the stator terminal voltage space vector to zero:

$$v_s(t) = 0 \quad \text{for all } t \geq 0$$

In the stationary d-q frame this is $v_{ds}(t) = v_{qs}(t) = 0$. In a three-wire positive-sequence d-q model, this is equivalent to zero line-to-line voltage at the motor terminals. The term “bolted” means the fault has zero impedance, producing the highest possible fault current and torque (conservative case for equipment rating). Whether the short point is grounded is not material unless zero-sequence paths are explicitly modeled.

8.2 The Fault as an Initial-Value Problem

The short-circuit transient is an **initial-value problem**: the system is in a known steady state at $t = 0^-$ (just before the fault), and we want to compute the evolution from $t = 0^+$ onward after the fault. The differential equations remain the same; only the applied voltage changes.

The initial conditions for the post-fault simulation are:

$$\psi_s(0^+) = \psi_s(0^-) = \psi_{s0}$$

$$\psi_r(0^+) = \psi_r(0^-) = \psi_{r0}$$

$$\omega_m(0^+) = \omega_m(0^-) = \omega_{m0}$$

where ψ_{s0} , ψ_{r0} , and ω_{m0} are computed from the pre-fault steady state.

8.3 Flux Continuity at Fault Inception

The principle of **flux continuity** is fundamental to electromagnetic transient analysis. It follows from Faraday's law:

$$v = \frac{d\psi}{dt} \implies \psi(t) = \psi(0) + \int_0^t v(\tau) d\tau$$

For the integral to be finite, $\psi(t)$ must be a continuous function of time - it cannot jump instantaneously. If it did, $d\psi/dt$ would be infinite, requiring infinite voltage.

Therefore, even though the voltage changes abruptly from V_{s0} to 0 at $t = 0$, the flux linkages ψ_s and ψ_r remain unchanged at the instant of the fault:

$$\psi_s(0^+) = \psi_s(0^-)$$

8.4 Why Flux and Current Are Continuous but Their Derivatives Change

Flux linkages are continuous at fault inception because an instantaneous flux jump would require infinite voltage (Faraday's law).

In this linear model, the inductance matrix does not change at the instant of fault. The currents are algebraic functions of the continuous flux linkages:

$$i_s(0^+) = \frac{L_r \psi_{s0} - L_m \psi_{r0}}{\Delta}, \quad i_r(0^+) = \frac{-L_m \psi_{s0} + L_s \psi_{r0}}{\Delta}$$

Since ψ_s , ψ_r , and all inductances are unchanged at $t = 0$, the currents i_s and i_r are also **continuous** across the fault:

$$i_s(0^-) = I_{s0} = i_s(0^+), \quad i_r(0^-) = I_{r0} = i_r(0^+)$$

What changes **discontinuously** is the applied voltage boundary condition (v_s goes from V_{s0} to 0). This causes the **derivatives** $d\psi_s/dt$, $d\psi_r/dt$ to jump at $t = 0$, but the fluxes and currents themselves remain continuous. The distinction matters: current continuity follows from flux continuity in a constant-parameter linear model.

9. The DC Offset

9.1 Origin of the DC Offset

The **DC offset** (also called the **asymmetrical component** or **DC transient**) in the stator current arises from the flux-continuity requirement.

Consider the stator flux at $t = 0^+$:

$$\psi_s(0^+) = \psi_{s0} = L_s I_{s0} + L_m I_{r0}$$

At $t \rightarrow \infty$ (after all transients have decayed), the flux must approach zero because there is no applied voltage to sustain it:

$$\lim_{t \rightarrow \infty} \psi_s(t) = 0$$

The transition from ψ_{s0} to 0 is not smooth. The stator flux ψ_s in the stationary frame can be decomposed into:

- A **synchronous component** ψ_s^{ac} rotating at ω_s , which decays with the rotor short-circuit time constant $T_{r,sc}$.
- A **DC component** ψ_s^{dc} fixed in the stationary frame, which decays with the stator DC time constant $T_{s,dc}$.

The DC component is the mathematical expression of flux continuity: the initial flux ψ_{s0} must be supported by a combination of AC and DC components whose sum at $t = 0$ equals ψ_{s0} .

9.2 Mathematical Description

The analytic solution for the stator flux during a three-phase short circuit (simplified, neglecting speed dynamics) is:

$$\psi_s(t) = \psi_{s0}e^{-t/T_{s,dc}} + \psi_s^{ac}(t)$$

where the AC component satisfies the homogeneous equation with the rotor flux decay.

The corresponding stator current is:

$$i_s(t) = \frac{L_r}{\Delta}\psi_s(t) - \frac{L_m}{\Delta}\psi_r(t)$$

The DC component in ψ_s produces a DC component in i_s that decays as $e^{-t/T_{s,dc}}$. This is what the **quick calculation's** `DC_OFFSET_FACTOR` attempts to approximate - but the d-q model captures it exactly through the state equations.

9.3 DC Offset Distribution Among Phases

The fault inception angle θ_0 determines how the DC offset is distributed among the three physical phases, but it does not affect the total magnetic energy stored at the instant of fault. For the ideal balanced model, the initial flux and current space vectors all rotate together by $e^{j\theta_0}$, and the scalar torque is invariant to this common rotation (see Section 10.5).

9.4 Decay of the DC Component

The DC component decays exponentially with the **stator DC time constant**:

$$T_{s,dc} = \frac{L_{s,sc}}{R_s}$$

where $L_{s,sc}$ is the short-circuit inductance seen from the stator:

$$L_{s,sc} = L_{\ell s} + \frac{L_m L_{\ell r}}{L_m + L_{\ell r}}$$

For the test motor, $T_{s,dc} \approx 84.8$ ms. This means: - After 50 ms ($\approx 0.6 \cdot T_{s,dc}$), the DC offset has decayed to $e^{-0.6} \approx 55\%$ of its initial value. - After 200 ms ($\approx 2.36 \cdot T_{s,dc}$), the DC offset has decayed to $e^{-2.36} \approx 9.4\%$ of its initial value.

This explains the comparison results: the quick and d-q methods agree well for $t > 100$ ms (the DC offset has mostly decayed), but differ significantly for $t < 50$ ms (the DC offset is dominant).

9.5 Effect on the Torque Waveform

The torque waveform exhibits an asymmetric shape (unequal positive and negative peaks) because of the multi-component nature of the transient. The waveform can be interpreted as the superposition of transient components arising from stator and rotor flux interactions with distinct decay rates and oscillation frequencies. (This decomposition into separate DC and AC components, e.g. $\psi_s = \psi_s^{dc} + \psi_s^{ac}$, is an approximate interpretation; the rigorous description is that the transient is a superposition of the system's natural eigenmodes identified in Section 5.2.)

The quick calculation, using a single-frequency cosine model, cannot capture this asymmetry without the optional `DC_OFFSET_FACTOR` term.

Note on spectral content: The exact frequency components should be verified by FFT analysis of the simulated waveform rather than assumed from heuristic labels. See Section 15.3 for an FFT-based spectral analysis of the test motor torque waveform.

10. The Fault Inception Angle

10.1 Physical Meaning of theta0

The **fault inception angle** θ_0 is the instantaneous phase angle of the a -phase voltage at the moment the fault occurs:

$$v_{as}(t) = \sqrt{2}V_{ph} \cos(\omega_s t + \theta_0)$$

At $t = 0$:

$$v_{as}(0) = \sqrt{2}V_{ph} \cos(\theta_0)$$

$$v_{bs}(0) = \sqrt{2}V_{ph} \cos(\theta_0 - 120^\circ)$$

$$v_{cs}(0) = \sqrt{2}V_{ph} \cos(\theta_0 - 240^\circ)$$

In the stationary d-q frame:

$$V_{s0} = \sqrt{2}V_{ph}e^{j\theta_0} = \sqrt{2}V_{ph}(\cos \theta_0 + j \sin \theta_0)$$

θ_0 is set by the INITIAL_VOLTAGE_ANGLE_DEG parameter in `input.jsonc` (line 51). In a real power system, the fault inception angle is essentially random - the fault can occur at any point on the voltage waveform.

10.2 Effect on Initial Conditions

The inception angle affects the initial conditions through the applied voltage space vector V_{s0} . Expanding the 2x2 system:

$$\begin{bmatrix} R_s + j\omega_s L_s & j\omega_s L_m \\ js\omega_s L_m & R_r + js\omega_s L_r \end{bmatrix} \begin{bmatrix} I_{s0} \\ I_{r0} \end{bmatrix} = \begin{bmatrix} \sqrt{2}V_{ph}(\cos \theta_0 + j \sin \theta_0) \\ 0 \end{bmatrix}$$

Solving this gives I_{s0} , I_{r0} , and consequently ψ_{s0} , ψ_{r0} as functions of θ_0 . The key point is that **the magnitudes** $|I_{s0}|$, $|I_{r0}|$, $|\psi_{s0}|$, $|\psi_{r0}|$ **are independent of** θ_0 . Only the **phase angles** of these quantities change with θ_0 .

This means the pre-fault torque T_{nom} , which depends only on the magnitudes, is also independent of θ_0 :

$$T_{nom}(\theta_0) = \text{constant}$$

10.3 No Effect on Electromagnetic Torque in the Ideal Balanced Model

In the ideal balanced model, the entire electromagnetic torque waveform is **exactly invariant** to θ_0 , not merely the peak absolute value. The reason is rotational symmetry: multiplying the pre-fault voltage by $e^{j\theta_0}$ rotates all initial flux and current space vectors by the same factor, and this common rotation cancels in the torque expression $T_e = -(3/2)p \operatorname{Im}\{\psi_s i_s^*\}$.

The DC offset does redistribute among the three physical phases as θ_0 varies, but the total torque - a scalar invariant under common rotation of all space vectors - is unchanged.

10.4 Periodicity: Why 180 deg

The transformation $abc \rightarrow dq$ involves $\cos \theta_0$ and $\sin \theta_0$ terms. Replacing θ_0 by $\theta_0 + 180^\circ$ negates the pre-fault voltage V_{s0} , which flips the signs of I_{s0} , I_{r0} , ψ_{s0} , ψ_{r0} . However, since torque depends on products of these quantities (e.g., $\psi_{ds} i_{qs}$), a simultaneous sign flip of all quantities leaves the torque unchanged. Therefore, the torque behaviour is 180° -periodic in θ_0 .

10.5 Why Torque Is Exactly Angle-Invariant for the Ideal Balanced Model

For the ideal balanced, linear, symmetric three-phase terminal short-circuit model, the electromagnetic torque waveform is **exactly invariant** to the fault inception angle θ_0 , not merely the peak absolute value. The proof follows from rotational symmetry.

Proof: The initial condition is $V_{s0} = \sqrt{2}V_{ph}e^{j\theta_0}$. The pre-fault system is linear, so I_{s0} , I_{r0} , ψ_{s0} , and ψ_{r0} are each proportional to $e^{j\theta_0}$. At every subsequent time step, the ODE system is rotationally symmetric: if $\{\psi_s(t), \psi_r(t)\}$ is a solution starting from $\theta_0 = 0$, then $\{e^{j\theta_0}\psi_s(t), e^{j\theta_0}\psi_r(t)\}$ is the solution starting from θ_0 . The torque is:

$$T_e = -\frac{3}{2}p \operatorname{Im}\{\psi_s i_s^*\}$$

where $i_s = (L_r\psi_s - L_m\psi_r)/\Delta$. Under the common rotation:

$$\psi'_s = e^{j\theta_0}\psi_s, \quad \psi'_r = e^{j\theta_0}\psi_r, \quad i'_s = e^{j\theta_0}i_s, \quad i'_r = e^{j\theta_0}i_r$$

$$T'_e = -\frac{3}{2}p \operatorname{Im}\{\psi'_s i'^{*}_s\} = -\frac{3}{2}p \operatorname{Im}\{e^{j\theta_0}\psi_s \cdot (e^{j\theta_0}i_s)^*\} = -\frac{3}{2}p \operatorname{Im}\{\psi_s i_s^*\} = T_e$$

Therefore the full torque trajectory $T_e(t)$ is identical for all θ_0 . The phase currents and d/q axis components redistribute with θ_0 , but the electromagnetic torque does not.

When angle dependence can appear: The invariance breaks if the model includes unbalanced faults, phase-asymmetric winding parameters, magnetic saturation with spatial saliency, non-sinusoidal winding distribution, zero-sequence paths, or supply/network asymmetry. The angle sweep in `scim_calc/sweep.py` serves as a validation routine confirming this invariance (and as infrastructure for future extensions).

10.6 Why the Quick Calculation Appears to Show Asymmetry

In the d-q model, the torque waveform is angle-invariant for the ideal balanced case. However, the **quick calculation** produces a symmetric waveform (equal positive and negative peaks) because it models only a single-frequency oscillation. The difference between the d-q model and the quick calculation (the latter underestimating the negative peak by ~10-11%) is **not** due to the inception angle, but due to the quick method's inability to capture the full multi-component flux interaction - this difference exists at all θ_0 equally.

For small slip (near synchronous speed), $(1-s)\omega_s \approx \omega_s$, so the torque components in the d-q model have nearly the same frequency, producing a waveform that appears closer to the quick calculation's single-frequency approximation. For larger slip, the frequency separation increases, and the d-q waveform's multi-component nature becomes more apparent.

10.7 Angle Sweep Implementation

For the ideal balanced model, the torque is exactly invariant to θ_0 , so the angle sweep in `scim_calc/sweep.py` functions as a **validation routine** - confirming that the simulation produces identical torque traces (within floating-point tolerance) for all inception angles. It also serves as infrastructure for future extensions where angle dependence may appear (unbalanced faults, saturation with saliency, etc.).

The sweep uses a two-phase approach for efficiency, detecting any deviation from invariance rather than identifying a maximum:

Phase 1 - Coarse sweep: - Range: 0° to 180° (the 180° periodicity). - Step: default 2° (configurable via `coarse_step`). - Resolution: reduced to `max(500, N_POINTS//10)` time steps over `min(0.1, T_END)` seconds. - Verifies that $\max(|T_e|)$ is constant across all angles (within numerical tolerance).

Phase 2 - Refinement (for future extensions): - Range: $\pm 4^\circ$ (configurable via `refine_width`) around any angle of interest. - Step: default 0.5° (configurable via `refine_step`). - Resolution: full (original `N_POINTS` and `T_END`). - Provides higher-resolution confirmation of invariance.

The two-phase approach reduces computation time by approximately 10x compared to a full-resolution sweep of all 180 angles.

Implementation: `scim_calc/sweep.py`, lines 18-108. The function `angle_sweep()` returns arrays of peak positive, negative, and absolute torque for each angle, allowing verification that all values are equal (within tolerance) for the current model.

Part V - Numerical Integration

11. The Runge-Kutta 4th-Order Method (RK4)

11.1 The General Runge-Kutta Framework

For an initial-value problem:

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0$$

Runge-Kutta methods approximate y_{n+1} from y_n by evaluating f at one or more intermediate points within the interval $[t_n, t_{n+1}]$.

An s -stage Runge-Kutta method is defined by the Butcher tableau:

$$\begin{array}{c|cccc} c_1 & a_{11} & a_{12} & \cdots & a_{1s} \\ c_2 & a_{21} & a_{22} & \cdots & a_{2s} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ c_s & a_{s1} & a_{s2} & \cdots & a_{ss} \\ \hline & b_1 & b_2 & \cdots & b_s \end{array}$$

where:

$$k_i = f \left(t_n + c_i h, y_n + h \sum_{j=1}^s a_{ij} k_j \right), \quad i = 1, \dots, s$$

$$y_{n+1} = y_n + h \sum_{i=1}^s b_i k_i$$

11.2 The RK4 Butcher Tableau

RK4 is a 4-stage, 4th-order explicit method with the tableau:

$$\begin{array}{c|cccc} 0 & 0 & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array}$$

The corresponding algorithm is:

$$\begin{aligned}
k_1 &= f(t_n, y_n) \\
k_2 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right) \\
k_3 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right) \\
k_4 &= f(t_n + h, y_n + hk_3) \\
y_{n+1} &= y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)
\end{aligned}$$

Implementation: `scim_calc/dq.py`, lines 92-97:

```

def rk4_step(state, dt):
    k1 = rhs(state)
    k2 = rhs(state + 0.5 * dt * k1)
    k3 = rhs(state + 0.5 * dt * k2)
    k4 = rhs(state + dt * k3)
    return state + (dt / 6.0) * (k1 + 2.0 * k2 + 2.0 * k3 + k4)

```

11.3 Local Truncation Error

The **local truncation error** (LTE) of RK4 is the error introduced in a single step, assuming the previous step was exact:

$$\text{LTE} = y(t_{n+1}) - y_{n+1} = Ch^5 y^{(5)}(\xi) + \mathcal{O}(h^6)$$

for some $\xi \in [t_n, t_{n+1}]$, where C is a constant and $y^{(5)}$ is the 5th derivative of the exact solution.

For the induction machine system, the 5th derivative term involves products of the machine parameters and state variables. A conservative bound for the LTE in each state variable is:

$$|\text{LTE}| \lesssim \frac{h^5}{720} \max_{t \in [t_n, t_{n+1}]} \left| \frac{d^5 y}{dt^5} \right|$$

11.4 Global Error and Convergence Order

The **global error** (GE) at time T accumulates over $N = T/h$ steps:

$$\text{GE}(T) \leq \frac{T}{h} \cdot \max(\text{LTE}) \propto h^4$$

Thus RK4 is **4th-order convergent**: halving the step size reduces the global error by a factor of $2^4 = 16$.

11.5 Stability Region

The **absolute stability region** of RK4 is the set of $z = \lambda h$ (where λ is an eigenvalue of the system matrix) for which the numerical solution remains bounded. For RK4 applied to the test equation $y' = \lambda y$:

$$y_{n+1} = R(z)y_n, \quad R(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}$$

The stability region boundary is $|R(z)| = 1$. For real λ (purely dissipative modes), the stability interval is approximately $[-2.78, 0]$. For complex λ (oscillatory modes), the region extends further into the left half-plane.

The RK4 stability check must use the **full complex eigenvalues** of the linearized 4-state electrical system (not just the damping rates). For the test motor, the eigenvalues are complex conjugates with magnitudes dominated by $\omega_s = 2\pi \cdot 60 \approx 377$ rad/s. With a step size of $h = 40 \mu\text{s}$:

$$|z| = |\lambda|h \approx 377 \times 4 \times 10^{-5} = 0.015$$

This is well within the stability region ($|z| \ll 2.78$), so the integration is numerically stable with a comfortable margin.

11.6 Step Size Selection for This Application

The step size is determined by:

$$h = \frac{T_{END}}{N_{POINTS} - 1}$$

With $T_{END} = 0.2$ s and $N_{POINTS} = 5000$:

$$h = \frac{0.2}{4999} \approx 4.0 \times 10^{-5} \text{ s} = 40 \text{ } \mu\text{s}$$

This step size satisfies the following criteria:

1. **Expected accuracy:** With $h \approx 40 \text{ } \mu\text{s}$, RK4 is expected to provide high accuracy for the 60 Hz electrical transient and the dominant exponential decays. The 4th-order convergence rate means halving the step size reduces the global error by a factor of 16. Accuracy should be verified using a step-halving convergence test (see Section 11.8).
2. **Stability:** As shown above, $|\lambda|h \approx 0.015$, well within the RK4 stability region.
3. **Waveform resolution:** The 60 Hz fundamental has a period of 16.7 ms. With $h = 40 \text{ } \mu\text{s}$, there are ≈ 417 steps per cycle, providing excellent resolution of the torque waveform.

11.7 Implementation Structure in dq.py

The integration loop in `dq.py` follows this structure:

1. Create time vector `t = linspace(0, T_END, N_POINTS)`
2. Compute `dt = t[1] - t[0]`
3. Allocate output arrays (`T_fault`, `omega_m_trace`, `Is_abs`)
4. Set `state = initial condition y0`
5. For each time step `k`:
 - a. Compute torque and current at current state
 - b. Store in output arrays
 - c. If not the last step, advance state with `rk4_step`

The key optimization is that the torque and current are computed **at the beginning of each step** (after the state has been updated), so the output values correspond to the time points exactly. The state is advanced to the next time point using the current state's derivative information.

12. Alternative Numerical Methods

12.1 Forward Euler: The Simplest

$$y_{n+1} = y_n + hf(t_n, y_n)$$

- **Order:** 1st-order accurate (global error $\mathcal{O}(h)$).
- **Stability:** $|1 + \lambda h| < 1$, giving $-2 < \lambda h < 0$ for real λ .
- **Pros:** Simplest possible, minimal operations per step.
- **Cons:** Poor accuracy requires very small step sizes; instability at moderate step sizes.

For this system, Euler would require $\lambda h < 2$, giving $h < 2/12 \approx 0.17$ s. This is satisfied, but the accuracy would be poor: with 5000 steps, the global error would be $\mathcal{O}(h) \approx \mathcal{O}(4 \times 10^{-5})$, compared to $\mathcal{O}(h^4) \approx \mathcal{O}(10^{-18})$ for RK4.

12.2 Trapezoidal Integration (Crank-Nicolson)

$$y_{n+1} = y_n + \frac{h}{2} [f(t_n, y_n) + f(t_{n+1}, y_{n+1})]$$

- **Order:** 2nd-order accurate.

- **Stability:** A-stable (the stability region is the entire left half-plane). This means trapezoidal integration is stable for any step size for stable systems.
- **Pros:** Unconditionally stable; good accuracy.
- **Cons:** Implicit - requires solving a system of equations at each step. For the nonlinear induction-machine ODE, this would require Newton iteration at each step.

Trapezoidal integration is widely used in electromagnetic transient programs (EMTP-type) for power system simulations. However, the nonlinearity from the $j\omega_{re}\psi_r$ term and the mechanical equation would require Newton iterations, adding significant complexity per step.

12.3 Runge-Kutta 45 (Adaptive)

RK45 (also known as **ode45** or the **Dormand-Prince** method) uses a 5th-order formula with a 4th-order embedded formula to estimate the local error and adjust the step size automatically.

- **Order:** 5th-order (global error $\mathcal{O}(h^5)$).
- **Pros:** Automatic step-size control; excellent accuracy; no tuning required.
- **Cons:** More stages per step (6 evaluations of f vs 4 for RK4); more complex implementation.

For this application, the ODE is smooth and the dynamics are well-understood, so fixed-step RK4 is adequate. Adaptive stepping would be beneficial if the system had rapid transients followed by slow decay (which it does, but the decay is exponential and well-resolved by fixed steps).

12.4 Backward Differentiation Formulas (BDF/Gear)

BDF methods (also called **Gear's methods**) are implicit multi-step methods designed for stiff systems.

- **Order:** Variable (1st to 6th).
- **Stability:** A-stable (BDF1, BDF2) or nearly A-stable (BDF3-BDF6).
- **Pros:** Excellent for stiff systems; large step sizes possible.
- **Cons:** Implicit (require Newton iteration); multi-step (require starting procedure); complex implementation.

BDF methods are the standard for circuit simulation (SPICE) and power system transient stability. However, they are overkill for a system with a stiffness ratio of only ≈ 18 , which is well within the capability of explicit RK4.

12.5 Comparison Summary

Method	Order	Steps per eval	Stability limit	Implicit?	Complexity
Forward Euler	1st	1	$\lambda h < 2$	No	Minimal
Trapezoidal	2nd	2	A-stable	Yes	Moderate (Newton)
RK4	4th	4	$\lambda h < 2.78$	No	Low
RK45 (adaptive)	5th	6	Good	No	Moderate
BDF2	2nd	1 per Newton iter	A-stable	Yes	High (Newton + startup)
BDF4	4th	1 per Newton iter	Near A-stable	Yes	High

12.6 Why RK4 Was Chosen

1. **Sufficient accuracy:** 4th-order convergence with 5000 steps over 0.2 s gives far more accuracy than needed for engineering purposes.
2. **Sufficient stability:** The stiffness ratio ($S \approx 18$) is far below the threshold where explicit methods struggle ($S > 100$ typically).
3. **Self-starting, no iterations required:** Unlike implicit methods, RK4 does not require solving nonlinear equations.
4. **Simple implementation:** The code is 6 lines and trivially verifiable.
5. **Fixed step size is adequate:** The dynamics are smooth and the time constants are well-separated, so adaptive stepping offers no advantage.
6. **Industry standard:** RK4 is the most widely used numerical integration method in science and engineering. It is well-understood, well-tested, and familiar to most engineers.

Part VI - Output and Post-Processing

13. Torque Computation at Each Step

13.1 Flux-Current Inversion

At each time step, the stator and rotor currents are recovered from the flux linkages by inverting the 2x2 inductance matrix:

```
i_s = (Lr * psi_s - Lm * psi_r) / Delta
i_r = (-Lm * psi_s + Ls * psi_r) / Delta
```

This is called from two places: 1. Inside `rhs()` (to compute $d\psi_s/dt$ and $d\psi_r/dt$). 2. In the main time-stepping loop (to compute torque for the output trace).

While this duplicates the i_s computation (it is computed once in `rhs()` and again at the beginning of the next step in the main loop), the cost is negligible for a 2x2 matrix inversion (2 multiplications, 1 addition, 1 division per current).

13.2 Torque from Space Vectors

The torque is computed using:

$$T_e = -\frac{3}{2}p \operatorname{Im}\{\psi_s i_s^*\}$$

This is the instantaneous electromagnetic torque. It is computed at the beginning of each time step, before the state is advanced.

13.3 Sign Convention and Normalization

The pre-fault torque T_{nom} is computed from the initial conditions:

```
T_nom_raw = torque_from_flux_current(psi_s0, Is0, pole_pairs)
```

This is a signed quantity. The sign convention is:

```
torque_sign = 1.0 if T_nom_raw >= 0.0 else -1.0
T_fault[k] = torque_sign * Te_raw
```

This ensures that the output torque T_{fault} is positive at $t = 0$ (motoring convention), and the sign indicates the instantaneous torque direction relative to the pre-fault direction.

13.4 Numerical Considerations

The torque computation involves: - **Complex conjugate:** $\psi_s i_s^* = (\psi_{ds} + j\psi_{qs})(i_{ds} - ji_{qs})$ - 2 real multiplications and 2 real additions. - **Imaginary part:** extract the coefficient of j - trivial. - **Multiplication by $-3p/2$:** 1 real multiplication.

The total operation count is approximately 5-10 floating-point operations per time step for the torque computation.

The torque values stored in the output array are in N · m. The percentage relative to nominal is computed during CSV export:

```
T_pct = 100.0 * d["T_fault"] / d["T_nom"]
```

14. Saved Outputs

14.1 CSV Format for the d-q Model

The d-q model output is saved to `data/dq_scim_short_circuit.csv` with columns:

Column	Description	Unit	Type
time_s	Time from fault inception	s	float
T_fault_Nm	Electromagnetic torque (signed)	N · m	float
T_fault_over_T_nom_percent	Torque as % of nominal	%	float
omega_m_rad_per_s	Mechanical rotor speed	rad/s	float
abs_i_s_peak_A	Peak stator current magnitude	A	float

The CSV uses scientific notation for all values (default `numpy.savetxt` formatting) with a header row. The `comments=""` argument ensures no `#` prefix on the header.

Implementation: `run.py`, lines 94-104 (`_save_dq()`).

14.2 Comparison with the Quick Calculation

The quick calculation output is saved to `data/quick_scim_short_circuit.csv` with columns:

Column	Description	Unit
time_s	Time from fault inception	s
T_fault_Nm	Signed torque estimate	N · m
T_fault_over_T_nom_percent	Torque as % of nominal	%
T_envelope_Nm	Positive envelope	N · m
T_envelope_over_T_nom_percent	Envelope as % of nominal	%

14.3 Comparison Metrics

The comparison module (`scim_calc/compare.py`) computes:

Metric	Description
T_nom	Nominal torque (should be identical between methods)
quick_pos_peak / dq_pos_peak	Maximum positive torque
quick_neg_peak / dq_neg_peak	Minimum (most negative) torque
diff_rms	RMS of the pointwise difference
diff_rms_pct	RMS difference as % of T_nom
diff_max	Maximum absolute pointwise difference
diff_early_rms_pct	RMS difference for $t \leq 50$ ms
diff_late_rms_pct	RMS difference for $t \geq 100$ ms

The time-segmented metrics are valuable because the two methods agree well in the late transient (after the DC offset has decayed) but differ significantly in the early transient. This confirms that the primary discrepancy is the DC offset effect.

Implementation: `scim_calc/compare.py`, lines 11-59 (`compute_comparison()`).

14.4 Overlay Plot

The overlay plot (`data/scim_short_circuit_overlay.png`) shows both torque traces on the same axes:

- Quick calculation: dashed line
- d-q model: solid line
- Nominal torque: dotted horizontal line at 100%

This plot provides a visual comparison of the two methods' waveforms. The asymmetry of the d-q trace (larger negative peaks) and the symmetry of the quick trace are immediately visible.

Part VII - Validation and Analysis

15. Comparison of Methods: Quick vs d-q

15.1 Theoretical Differences

Aspect	Quick Calculation	d-q Model
Model type	Closed-form analytical	State-space time-domain
Torque spectrum	Single frequency (ω_T)	Full multi-component spectrum
DC offset	Optional additive term (DC_OFFSET_FACTOR)	Natural (via flux continuity)
Torque waveform	Symmetric (equal pos/neg peaks)	Asymmetric (larger negative peak)
Decay model	Single exponential (e^{-t/T_r})	Multi-exponential (from eigenvalue spectrum)
Speed dynamics	Not available	Optional
Point-on-wave	Approximate ($FAULT_ANGLE_DEG$)	Exact (via θ_0 in initial conditions)
Computational cost	approximately 1 ms	approximately 500 ms (5000 steps)
Numerical method	N/A (direct formula)	RK4 integration
Saturation model	None	None
Parameter form	Reactances (X_s, X_r, X_m)	Inductances (L_s, L_r, L_m)

15.2 Quantified Differences for the Test Motor

For the 2000 HP, 4 kV, 60 Hz, 2-pole test motor (torque is invariant to θ_0 for the ideal balanced model - see Section 10.5):

Metric	Quick	d-q	Difference
T_{nom}	3989 N · m	3989 N · m	0%
Positive peak	+305.7% T_n	+299.6% T_n	+2.0%
Negative peak	-376.3% T_n	-416.8% T_n	-10.8%
Peak $ T $	376.3% T_n	416.8% T_n	-10.8%
Angle dependence	None (symmetric waveform)	None (exactly invariant for balanced model)	—
RMS difference (full)	-	-	12.8% T_n
RMS diff ($t < 50$ ms)	-	-	23.7% T_n
RMS diff ($t > 100$ ms)	-	-	2.5% T_n
Max pointwise diff	-	-	51.0% T_n

The quick calculation underestimates the worst-case negative torque by **10.8%** relative to the d-q model. This is a significant difference for equipment rating studies, where the worst-case torque determines shaft stress and coupling requirements.

15.3 FFT Spectral Analysis of the Test Motor Torque Waveform

The FFT of the simulated torque waveform (computed using `scim_calc/spectral.py`) reveals the dominant frequency components. Figure 2 shows the single-sided amplitude spectrum.

!FFT spectrum figure omitted: ../data/sweep_fft_spectrum.png was not available during PDF generation.

Figure 2: FFT amplitude spectrum of $T_{fault}(t)$ for the test motor (2000 HP, 4 kV, 60 Hz). Dominant peaks are marked with dashed lines.

For the test motor, the two dominant spectral peaks are:

Peak	Frequency	Amplitude (% Tn)	Relative (% of max)	Interpretation
1	59.99 Hz	88.4% Tn	100%	Electrical fundamental — stator-dominated mode (see eigenvalue pair 1, Section 5.2)
2	5.00 Hz	16.4% Tn	18.6%	Low-frequency component associated with rotor flux transient decay

The ≈ 60 Hz peak corresponds to the stator-dominated eigenvalue pair $(-12.38 \pm j373.68 \text{ rad/s}, f \approx 59.5 \text{ Hz})$ identified in Section 5.2. This component carries the majority of the oscillatory torque energy and is the primary source of the torque ripple visible in the time-domain waveform.

The low-frequency peak near 5 Hz (limited by the FFT frequency resolution of $\Delta f \approx 5 \text{ Hz}$ for a 0.2 s window) captures the aperiodic decay component associated with the nearly critically-damped rotor mode $(-11.79 \pm j0.37 \text{ rad/s})$. Its exact frequency content would require a longer simulation window to resolve fully, but its presence confirms the multi-component nature of the transient.

Note: The specific peak amplitudes depend on the motor parameters and operating point. The FFT analysis should be re-run whenever the motor or operating conditions change. The `scim_calc/spectral.py` module provides the `compute_fft()` and `find_peaks()` utilities for this purpose.

15.4 When Each Method Is Appropriate

Use the quick calculation when: - Performing rapid screening of many operating points or motor sizes. - The DC offset effect is known to be small (high-inertia loads, small X/R ratio). - Only order-of-magnitude accuracy is required ($\pm 20\%$). - The study requires negligible computation time.

Use the d-q model when: - Accurate peak torque values are needed for equipment rating. - The asymmetry between positive and negative peaks matters. - Point-on-wave effects are being studied. - The motor has a high X/R ratio (large DC offset). - Speed dynamics during the transient may affect the result. - The study results will be used for design decisions (shaft sizing, coupling selection, protection settings).

15.5 Verification and Regression Tests

The project includes a test suite that validates the key mathematical properties asserted in this document. The following table summarizes the available tests and what they verify:

Test file	Tests	What it validates
<code>test_invariance.py</code>	4	Fault inception angle has no effect on torque (rotational symmetry proof) at $0^\circ, 45^\circ, 90^\circ, 180^\circ$ and multiple angles
<code>test_current_continuity.py</code>	2	Stator and rotor currents are continuous across the fault instant (flux continuity + constant inductance matrix)
<code>test_torque_sign.py</code>	3	Pre-fault torque is positive (motoring convention); complex torque matches IEEE phasor torque
<code>test_eigenvalue_stability.py</code>	4	System matrix eigenvalues have negative real parts; RK4 stability condition $ R(z) < 1$ is satisfied with margin; eigenvalue magnitudes are in the expected range

Test file	Tests	What it validates
<code>test_spectral.py</code>	4	Dominant torque spectral peak is near 60 Hz; low-frequency content is present; no unexpected high-frequency numerical noise
<code>test_rk4_convergence.py</code>	2	Peak torque values converge as step size decreases; the default 5000-point resolution gives results within 0.1% of 10000-point resolution

Convergence behaviour (RK4): Running the simulation at increasing resolutions shows rapid convergence:

N_POINTS	Time step Δt	Positive peak (% T _n)	Negative peak (% T _n)	Peak (abs T, % T _n)
500	401 μs	299.61	-416.84	416.84
1,000	200 μs	299.59	-416.84	416.84
2,000	100 μs	299.58	-416.84	416.84
5,000	40 μs	299.62	-416.84	416.84

The default setting of 5000 points over 0.2 s ($\Delta t \approx 40 \mu s$) provides excellent resolution: the peak torque values vary by less than 0.05% even when the step size is increased by 10x.

Running the tests:

```
# Run all tests (excluding the slow convergence check)
python -m pytest tests/ -k "not slow"
```

```
# Run all tests including convergence
python -m pytest tests/
```

All tests are located in the `tests/` directory and use the fixture data in `tests/fixtures/` as regression baselines.

16. Parameter Sensitivity Analysis

16.1 Stator Resistance R_s

R_s affects: - The **stator DC time constant** $T_{s,dc} = L_{s,sc}/R_s$. Higher R_s reduces $T_{s,dc}$, causing the DC offset to decay faster. - The **pre-fault torque** T_{nom} through the $I_s^2 R_s$ loss. - The **initial short-circuit current** $I_{sc,ac0}$.

Sensitivity: For the test motor, a $\pm 10\%$ change in R_s changes the peak $|T|$ by approximately $\pm 2\%$.

16.2 Rotor Resistance R_r

R_r affects: - The **rotor short-circuit time constant** $T_{r,sc} = L_{r,sc}/R_r$. Higher R_r reduces $T_{r,sc}$, accelerating the decay of the AC torque component. - The **pre-fault slip** (through the torque-slip relationship). - The **peak torque magnitude** (higher R_r increases damping).

Sensitivity: A $\pm 10\%$ change in R_r changes the peak $|T|$ by approximately $\pm 5\%$. R_r is one of the most sensitive parameters.

16.3 Leakage Reactances X_s, X_r

X_s and X_r (or equivalently $L_{\ell s}$, $L_{\ell r}$) affect: - The **short-circuit reactances** $X_{s,sc}$, $X_{r,sc}$. - The **initial short-circuit current** (higher leakage $\rightarrow$ lower current). - The **torque oscillation frequency** (leakage affects the rotor frequency through the slip).

Sensitivity: A $\pm 10\%$ change in $X_s + X_r$ changes the peak $|T|$ by approximately $\mp 6\%$ (inverse relationship: higher leakage $\rightarrow$ lower torque).

16.4 Magnetizing Reactance X_m

X_m affects: - The **magnetizing current** (lower $X_m \rightarrow$ higher magnetizing current). - The **air-gap flux** and thus the torque capability. - The **coupling** between stator and rotor (the leakage coefficient σ).

Sensitivity: A $\pm 10\%$ change in X_m changes the peak $|T|$ by approximately $\mp 3\%$. Large changes in X_m (e.g., saturation effects) have a bigger impact.

16.5 Inertia J

J only matters when speed dynamics are enabled. With speed dynamics: - Low J allows the rotor to decelerate significantly during the transient, reducing the slip and the peak torque. - High J (or infinite J , i.e., constant speed) produces the highest possible peak torque.

For worst-case analysis, $J = \infty$ (constant speed) is the conservative choice. The default configuration uses this approach.

16.6 Fault Inception Angle θ_0

For the ideal balanced model, θ_0 has **no effect** on the electromagnetic torque waveform - the entire trace $T_e(t)$ is exactly invariant. The sweep results confirm this: the peak absolute torque is $416.84\% T_n$ at every angle to within floating-point precision. The invariance follows from the rotational symmetry of the model (see Section 10.5).

Angle dependence may appear in extended models that include unbalanced faults, saturation with spatial saliency, or non-sinusoidal winding distributions.

Part VIII - Extensions and Alternatives

17. Assumptions and Limitations

17.1 Linear Magnetics (No Saturation)

Assumption: All inductances ($L_{\ell s}$, $L_{\ell r}$, L_m) are constant, independent of current.

Reality: In ferromagnetic materials, the incremental permeability decreases at high flux densities. During a short circuit, the stator current can reach 5-10 times rated value, and the flux may push the iron into saturation.

Effect on results: Saturation reduces L_m at high currents, which: - Reduces the peak torque (less flux available for torque production). - Reduces the time constants (lower $L \rightarrow$ faster decay). - Changes the current waveshape (flattened peaks from saturation).

Modelling alternatives: - **Piecewise-linear saturation:** Approximate the $B-H$ curve with a piecewise-linear $i_m-\psi_m$ relationship. L_m takes two values: unsaturated (normal) and saturated (fault). - **Exponential saturation:** Use a smooth function like $\psi_m = a \tanh(b i_m)$ or $\psi_m = a i_m / (1 + b |i_m|)$. - **Look-up table:** Use manufacturer-provided saturation data.

Why not implemented: Saturation modelling requires the saturation characteristic, which is not available from standard motor nameplate data. The linear model provides a conservative (higher torque) estimate, which is appropriate for worst-case analysis.

17.2 Balanced Fault Only

Assumption: The fault is a balanced three-phase bolted short circuit at the motor terminals. In the positive-sequence d-q model, this is equivalent to zero line-to-line voltage at the terminals: the stator voltage space vector is forced to zero. Whether the short point is grounded is not material unless zero-sequence paths are explicitly modelled.

Reality: Power system faults can be: - **Line-to-line (L-L):** Two phases shorted together (not modelled). - **Line-to-ground (L-G):** One phase shorted (not modelled). - **Double line-to-ground (L-L-G):** Two phases shorted (not modelled). - **Three-phase (L-L-L):** All three phases shorted - this is the modelled case.

Why not implemented: Unbalanced faults require sequence-component analysis or a full abc -frame model. The d-q model in the stationary frame implicitly assumes balanced operation. Extending to unbalanced faults would require either:

- A full three-phase (abc) model with six coupled differential equations (three stator, three rotor).
- A sequence-component approach with positive, negative, and zero-sequence networks.

17.3 Single-Cage Rotor

Assumption: The rotor can be represented by a single equivalent circuit (resistance R_r , leakage inductance $L_{\ell r}$).

Reality: Large induction motors often have **double-cage** or **deep-bar** rotors, which provide different characteristics during starting (high resistance, high torque) and running (low resistance, high efficiency). These rotors have two effective time constants - a fast one for the starting cage and a slow one for the running cage.

Modelling approach: A double-cage rotor adds an additional rotor circuit:

$$\psi_{r1} = L_m i_s + L_{r1} i_{r1} + L_{r12} i_{r2}$$

$$\psi_{r2} = L_m i_s + L_{r12} i_{r1} + L_{r2} i_{r2}$$

This increases the model order by two (two additional flux states). The parameters (R_{r1} , R_{r2} , $L_{\ell r1}$, $L_{\ell r2}$, L_{r12}) are typically not available from standard data sheets.

17.4 Constant Rotor Temperature

Assumption: R_r is constant.

Reality: Rotor resistance increases with temperature. Copper has a temperature coefficient of approximately $0.00393 \Omega/\Omega/^\circ\text{C}$. A 50°C temperature rise increases R_r by approximately 20%.

Effect: Higher R_r would accelerate the decay of the rotor flux (reducing $T_{r,sc}$), reducing the duration and peak of the torque transient. The constant-resistance assumption is conservative (predicts higher torque).

17.5 No Skin Effect

Assumption: Current distribution in the rotor bars is uniform (frequency-independent resistance and leakage).

Reality: At high frequencies, rotor current tends to concentrate at the top of deep rotor bars (skin effect), increasing effective resistance and reducing effective leakage inductance. During a short-circuit transient, the rotor current contains components at multiple frequencies, including components at near-synchronous frequency arising from the stator DC offset (which is stationary relative to the stator and therefore seen by the rotor at approximately rotor electrical speed, near line frequency for a near-synchronous 2-pole motor).

Effect: The single-cage constant-parameter model may underestimate damping of high-frequency rotor current components. For deep-bar or double-cage rotors, the frequency-dependent rotor resistance and leakage may affect the early transient torque. The model should therefore be treated as an approximation for such machines.

17.6 Ideal Voltage Source Behind the Fault

Assumption: Before the fault, the motor terminals are connected to an ideal voltage source with zero impedance. The fault is at the motor terminals.

Reality: The motor is connected through cables, transformers, and the upstream network, all of which have impedance. A fault may not be exactly at the motor terminals - it could be on the feeder cable or within the motor itself.

Effect: Additional impedance between the source and the fault reduces the short-circuit current and torque. The assumption of a bolted fault at the terminals is the most conservative case.

17.7 No Torsional Dynamics

Assumption: The mechanical system is a single rigid inertia (rotor + load).

Reality: The shaft system between the motor and the load has torsional natural frequencies. A sudden torque transient can excite torsional oscillations, which if close to a natural frequency, can produce shaft torques significantly higher than the electromagnetic torque computed here.

Effect: This is a separate analysis typically performed using **multi-mass torsional models** with spring-mass-damper elements. The electromagnetic torque computed by this model serves as the **input** to the torsional analysis.

18. Possible Extensions

18.1 Magnetic Saturation Modelling

To include saturation:

Approach 1 - Variable magnetizing inductance:

```
def Lm_sat(psi_m):
    """Saturated magnetizing inductance."""
    psi_mag = abs(psi_m)
    if psi_mag < psi_threshold:
        return Lm_unsat
    else:
        return Lm_unsat * (psi_threshold / psi_mag) ** alpha
```

where $\psi_m = \psi_s - L_{\ell s} i_s$ is the air-gap flux, $\psi_{threshold}$ is the knee-point flux, and $\alpha \approx 0.5$ controls the saturation characteristic.

Approach 2 - Iterative flux-current solution: At each time step, after computing i_s, i_r from the linear flux-current inversion, recompute L_m based on ψ_m , then re-invert. Iterate until convergence.

This is computationally more expensive but physically more accurate.

18.2 Double-Cage or Deep-Bar Rotor

The state space expands from 4 flux states to 6:

$$\frac{d}{dt} \begin{bmatrix} \psi_s \\ \psi_{r1} \\ \psi_{r2} \end{bmatrix} = \begin{bmatrix} -R_s i_s \\ j\omega_{re} \psi_{r1} - R_{r1} i_{r1} \\ j\omega_{re} \psi_{r2} - R_{r2} i_{r2} \end{bmatrix}$$

The flux-current relation becomes:

$$\begin{bmatrix} \psi_s \\ \psi_{r1} \\ \psi_{r2} \end{bmatrix} = \begin{bmatrix} L_s & L_m & L_m \\ L_m & L_{r1} & L_{r12} \\ L_m & L_{r12} & L_{r2} \end{bmatrix} \begin{bmatrix} i_s \\ i_{r1} \\ i_{r2} \end{bmatrix}$$

This 3x3 system must be inverted at each time step.

18.3 Unbalanced Faults

A full *abc*-frame model has the form:

$$\frac{d}{dt} \begin{bmatrix} \psi_{abcs} \\ \psi_{abcr} \end{bmatrix} = \begin{bmatrix} \mathbf{v}_{abcs} - R_s \mathbf{i}_{abcs} \\ \mathbf{v}_{abcr} - R_r \mathbf{i}_{abcr} \end{bmatrix}$$

where the inductance matrix is 6×6 with rotor-position-dependent elements:

$$\mathbf{L}(\theta_r) = \begin{bmatrix} \mathbf{L}_s & \mathbf{L}_{sr}(\theta_r) \\ \mathbf{L}_{sr}^T(\theta_r) & \mathbf{L}_r \end{bmatrix}$$

This approach handles any fault type but is more complex and computationally expensive than the d-q model.

18.4 Supply Impedance and Voltage Sags

To model a fault upstream of the motor (not at the terminals), add the supply impedance Z_{source} between the ideal voltage source and the motor:

$$v_s(t) = V_{source}(t) - Z_{source} \cdot i_s(t)$$

For a fault at the motor terminals, $Z_{source} = 0$; for a fault on the feeder, Z_{source} depends on the cable impedance and the distance to the fault.

18.5 Inter-Turn and Winding Faults

These require a **winding-function** or **multiple-coupled-circuit** approach, where each stator coil or group of coils is modelled as a separate circuit with its own resistance, self-inductance, and mutual inductances to all other coils.

This is a specialized topic in machine diagnostics and is beyond the scope of this project.

18.6 Adaptive Time-Stepping

Replace fixed-step RK4 with an embedded Runge-Kutta pair (e.g., RK45 or **Bogacki-Shampine** RK23) that estimates the local error and adjusts the step size:

```
def rk45_step(state, dt):
    # Compute both 4th-order and 5th-order solutions
    y4, y5 = embedded_rk(state, dt)
    error = norm(y5 - y4)
    if error < tolerance:
        # Accept step, increase dt for next step
        return y5, min(2*dt, dt_max)
    else:
        # Reject step, decrease dt
        return state, max(dt/2, dt_min)
```

This would reduce computation time for smooth portions of the transient while maintaining accuracy during rapid changes.

18.7 Parallel Angle Sweep

The angle sweep is embarrassingly parallel. Using `concurrent.futures`:

```
from concurrent.futures import ProcessPoolExecutor

with ProcessPoolExecutor() as executor:
    results = list(executor.map(
        lambda a: run_dq_simulation(p, slip=slip, angle=a),
        angles
    ))
```

This would reduce the sweep time approximately linearly with the number of CPU cores.

Part IX - Appendices

19. Source Code Cross-Reference

19.1 scim_calc/ Package

Function / class	File	Lines	Purpose
load_jsonc()	config.py	16-23	Parse JSONC config file
normalize_params()	config.py	26-104	Compute derived parameters
currents_from_flux()	circuit.py	14-33	Invert flux-current relation
torque_from_flux_current()	circuit.py	36-44	Torque from space vectors
solve_prefault()	circuit.py	47-95	Pre-fault steady state
torque_from_phasors()	circuit.py	98-124	IEEE phasor torque
derive_slip()	circuit.py	127-174	Slip from nameplate power
run_dq_simulation()	dq.py	19-126	Main d-q simulation
rhs()	dq.py	72-89	ODE right-hand side
rk4_step()	dq.py	92-97	RK4 integration step
run_quick_calc()	quick.py	25-147	Quick analytical calculation
angle_sweep()	sweep.py	18-108	Two-phase angle sweep
compute_comparison()	compare.py	11-59	Compare results
print_comparison()	compare.py	62-76	Display comparison

19.2 Top-Level Files

Function	File	Lines	Purpose
main()	run.py	31-70	Entry point
_save_quick()	run.py	77-91	Save quick results
_save_dq()	run.py	94-104	Save d-q results
_save_comparison()	run.py	107-114	Save comparison
_plot_overlay()	run.py	133-150	Generate overlay plot

20. Glossary of Symbols

20.1 General

Symbol	Meaning	Unit
v, i, ψ	Instantaneous voltage, current, flux linkage	V, A, V · s
V, I, Ψ	RMS magnitude	V, A, V · s
$\bar{f}$	Complex space vector ($f_d + jf_q$)	-
f^*	Complex conjugate	-
$\text{Re}\{\cdot\}, \text{Im}\{\cdot\}$	Real and imaginary parts	-
ω_s	Electrical synchronous frequency	rad/s
f	Electrical frequency (Hz)	Hz
s	Slip (per-unit)	-
p	Number of pole pairs	-
θ_0	Fault inception angle	rad
j	Imaginary unit ($\sqrt{-1}$)	-

20.2 Machine Parameters

Symbol	Python key	Meaning	Unit
R_s	Rs	Stator resistance	Ω
R_r	Rr	Rotor resistance (referred to stator)	Ω
$X_s = \omega_s L_{\ell s}$	Xs	Stator leakage reactance at f	Ω

Symbol	Python key	Meaning	Unit
$X_r = \omega_s L_{\ell r}$	Xr	Rotor leakage reactance at f	Ω
$X_m = \omega_s L_m$	Xm	Magnetizing reactance at f	Ω
$L_{\ell s}$	Lls	Stator leakage inductance	H
$L_{\ell r}$	Llr	Rotor leakage inductance	H
L_m	Lm	Magnetizing inductance	H
$L_s = L_{\ell s} + L_m$	Ls	Stator self-inductance	H
$L_r = L_{\ell r} + L_m$	Lr	Rotor self-inductance	H
$\Delta = L_s L_r - L_m^2$	Delta	Leakage determinant	H ²
$\sigma = 1 - L_m^2 / (L_s L_r)$	-	Leakage coefficient	-
V_{LL}	V_LL	Line-line RMS voltage	V
V_{ph}	V_ph	Phase (line-neutral) RMS voltage	V
J	J	Total moment of inertia	kg · m ²
P_{mech}	P_mech	Mechanical output power	W

20.3 Time Constants

Symbol	Python key	Meaning	Unit
T_{r0}	Tr0	Rotor open-circuit time constant	s
$T_{r,sc}$	Tr_sc	Rotor short-circuit time constant	s
$T_{s,dc}$	Ts_dc	Stator DC time constant	s
$T_{decay,rotor}$	T_decay_rotor	Torque envelope (rotor-dominated)	s
$T_{decay,mixed}$	T_decay_mixed	Torque envelope (mixed)	s

20.4 Short-Circuit Quantities

Symbol	Python key	Meaning	Unit
$X_{s,sc}$	Xs_sc	Stator short-circuit reactance	Ω
$X_{r,sc}$	Xr_sc	Rotor short-circuit reactance	Ω
$I_{sc,ac,0}$	I_sc_ac0	Initial symmetrical RMS fault current	A
Ψ_{m0}	-	Initial air-gap flux (RMS)	V · s
$T_{env,0}$	T_env0	Initial torque envelope	N · m

21. References

1. **P. C. Krause, O. Wasynczuk, S. D. Sudhoff, and S. Pekarek**, *Analysis of Electric Machinery and Drive Systems*, 3rd ed. Wiley-IEEE Press, 2013. - The standard reference for d-q modelling of electric machines. Chapters 4 (Reference-Frame Theory) and 6 (Induction Machines) are directly relevant.
2. **I. Boldea and S. A. Nasar**, *The Induction Machine Handbook*. CRC Press, 2002. - Comprehensive treatment of induction machine theory, including transient analysis (Chapter 14) and fault conditions.
3. **IEEE Standard 112**, *IEEE Standard Test Procedure for Polyphase Induction Motors and Generators*. - The IEEE equivalent circuit and torque computation standard. Defines the per-phase equivalent circuit used in `circuit.py:torque_from_phasors()`.

4. **J. C. Butcher**, *Numerical Methods for Ordinary Differential Equations*, 3rd ed. Wiley, 2016. - The definitive reference on Runge-Kutta methods. The RK4 Butcher tableau and stability analysis in Section 11 follow this reference.
5. **U. Hairer, S. P. Nørsett, and G. Wanner**, *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2nd ed. Springer, 1993. - Comprehensive treatment of RK methods, including error analysis and stiffness classification.
6. **T. A. Lipo**, *Introduction to AC Machine Design*. Wiley-IEEE Press, 2017. - Modern treatment of machine design with transient analysis perspectives. Chapter 7 covers short-circuit torques.
7. **P. L. Alger**, *Induction Machines: Their Behavior and Uses*, 2nd ed. Gordon and Breach, 1970. - Classic text. Chapter 9 covers the short-circuit transient in detail, including the analytical solution for the DC offset.
8. **A. E. Fitzgerald, C. Kingsley, and S. D. Umans**, *Electric Machinery*, 6th ed. McGraw-Hill, 2003. - Standard undergraduate text. Chapter 6 covers induction machines; the d-q transformation is introduced in Chapter 3.
9. **N. Mohan, T. M. Undeland, and W. P. Robbins**, *Power Electronics: Converters, Applications, and Design*, 3rd ed. Wiley, 2003. - Reference for the amplitude-invariant vs power-invariant transformation conventions (Appendix A).
10. **H. W. Dommel**, *EMTP Theory Book*, 2nd ed. Microtran Power System Analysis Corporation, 1996. - The reference for electromagnetic transient simulation in power systems. The trapezoidal integration method (Section 12.2) is the EMTP standard.

Document version: 2.0 - July 2026

Corresponding code version: `scim_calc/` package as restructured July 2026

Author: Generated in collaboration with opencode AI assistant